

Basic Parsing Techniques

1. Define parsing. What is the role of a parser?

Ans: Parsing (also known as **syntax analysis**) can be defined as a process of analyzing a text which contains a sequence of tokens, to determine its grammatical structure with respect to a given grammar.

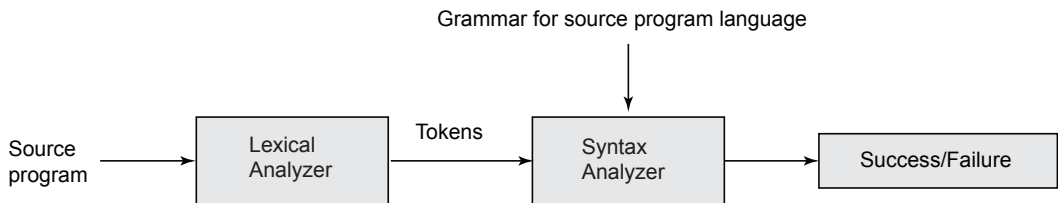


Figure 4.1 Parsing

Depending upon how the parse tree is built, parsing techniques are classified into three general categories, namely, *universal parsing*, *top-down parsing*, and *bottom-up parsing*. The most commonly used parsing techniques are top-down parsing and bottom-up parsing. Universal parsing is not used as it is not an efficient technique. The hierarchical classification of parsing techniques is shown in Figure 4.2.

Role of a parser: A parser receives a string of tokens from lexical analyzer and constructs a parse tree if the string of tokens can be generated by the grammar of the source language; otherwise, it reports the syntax errors present in the source string. The generated parse tree is passed to the next phase of the compiler, as shown in Figure 4.3.

The role of parser is summarized as follows:

- ❑ Perform context-free syntax analysis.
- ❑ Guides context-sensitive analysis.
- ❑ Generate an intermediate code.
- ❑ Report syntax errors in an intelligible manner.
- ❑ Attempts error correction.

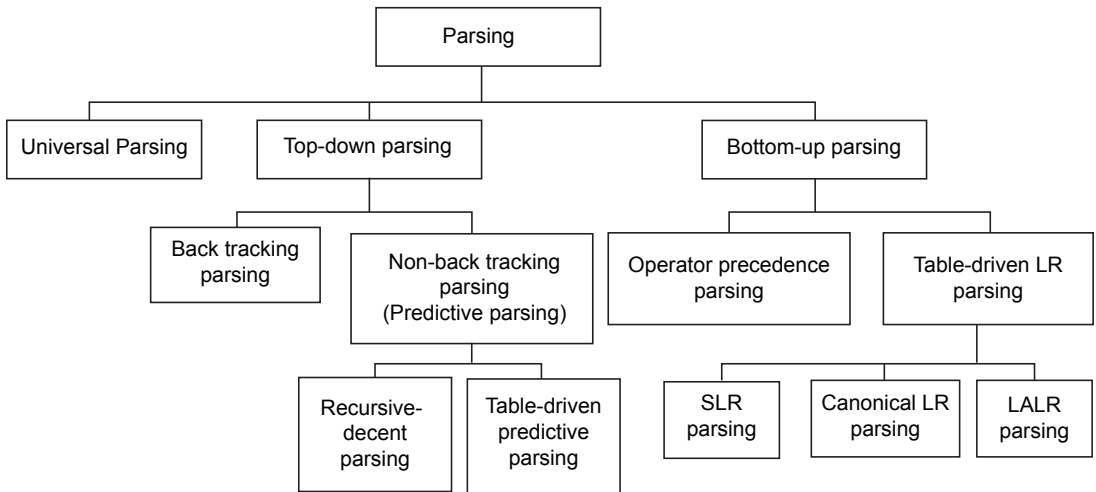


Figure 4.2 Classification of Parsing Techniques

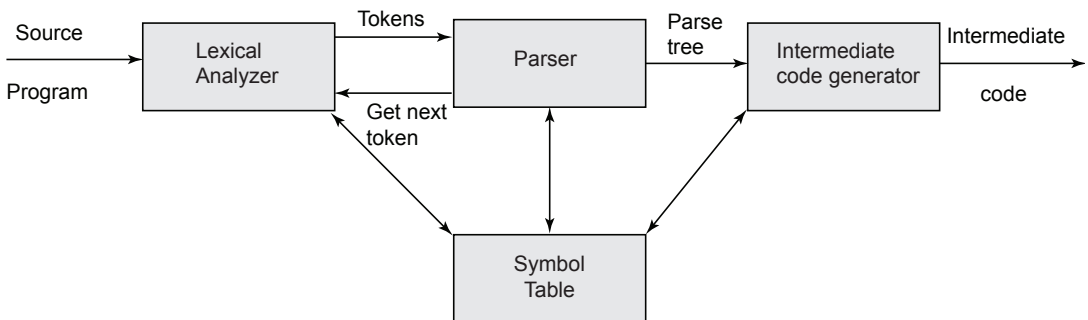


Figure 4.3 Position of a Parser in Compiler Model

2 Define parse tree. Construct the derivation of a parse tree.

Ans: A **parse tree** or **concrete syntax tree** is a tree data structure that is ordered and rooted and represents the syntactic structure (or grammatical structure) of a string. It can also be defined as a graphical representation of a derivation that shows the order in which grammar rules (productions) are applied to replace the non-terminals. A non-terminal S present in the head of the production is used to label the interior node and all of its children are labeled from left to right by the production symbols so that S can be replaced during the derivation.

Derivation of a parse tree: Derivations help in constructing a precise parse tree where productions are considered as rewriting rules. At each rewriting step in a derivation, a non-terminal is selected to be replaced and a production is taken for the derivational view with that non-terminal as head.

For example, consider the following grammar G :

$$S \rightarrow S + S \mid S * S \mid (S) \mid - S \mid a$$

The parse tree for $-(a + a)$ is shown in Figure 4.4. It can be written as $S \Rightarrow^* -(a + a)$, which implies that $-(a + a)$ can be derived by S in zero or more steps. Since S is a start symbol of grammar G , we say that $-(a + a)$ is a sentential form of G . A sentential form may contain both terminals and non-terminals, or may be empty.

To drive $-(a + a)$ from S , we start from the start symbol S of the grammar G and choose the production $S \rightarrow -S$. The replacement of a single S by $-S$ will be described by writing $S \Rightarrow -S$; then the production $S \rightarrow (S)$ can be applied to replace (S) by $(-S)$. Similarly, other productions can be applied to get a sequence of replacements to drive $-(a + a)$.

Thus, the derivation of $S \rightarrow -(a + a)$ can be written as

$$S \Rightarrow -S \Rightarrow -(S) \Rightarrow -(S + S) \Rightarrow -(a + S) \Rightarrow -(a + a).$$

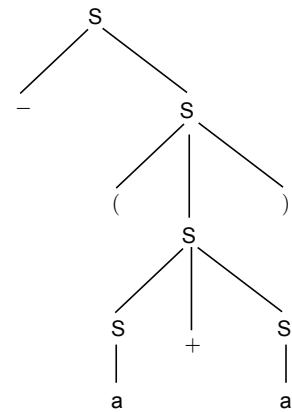


Figure 4.4 Parse Tree for $-(a + a)$

3. What is top-down parsing? Explain with the help of an example. Name the different parsing techniques used for top-down parsing.

Ans: Top-down parsing is a strategy to find the leftmost derivation of an input string. In top-down parsing, the parse tree is constructed starting from the root and proceeding toward the leaves (similar to a derivation), generating the nodes of the tree in preorder.

For example, consider the following grammar:

$$\begin{aligned} E &\rightarrow cDe \\ D &\rightarrow ab \mid a \end{aligned}$$

For the input string cae , the leftmost derivation is specified as:

$$E \Rightarrow cDe \Rightarrow cae$$

The derivation tree for the input string cae is shown in Figure 4.5.

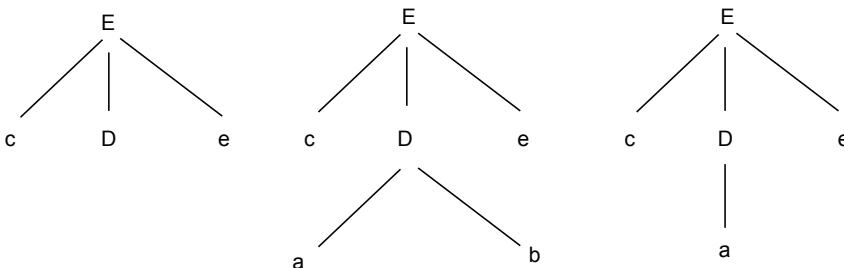


Figure 4.5 Parse Tree for $E \rightarrow cae$

Top-down parsing techniques:

- **Backtracking parsing:** In this technique, the parser initiates with the start symbol and applies the grammar productions until the target string is generated. The productions are applied according to the order in which they are specified in the grammar. If a choice leads to a dead end, then the parser backtracks to the last decision point, undo that decision, and try another production until the parser finds a suitable production for matching the whole input or it runs out of choices.

- **Non-backtracking parsing (Predictive parsing):** Predictive parsing does not require backtracking in order to derive the input string. Predictive parsing is possible only for the class of $LL(k)$ grammar (context-free grammar). The grammar should be free from left recursion and left factoring. Each non-terminal is combined with the next input signal to guide the parser to select the correct production rule that will lead the parser to match the complete input string.

There are two techniques of implementing top-down predictive parsers, namely, *recursive-decent* and *table-driven predictive parsings*.

4. Define recursive predictive parsing or predictive parsing.

Ans: It is a top-down parsing method, which consists of a set of mutually recursive procedures to process the input and handles a stack of activation records explicitly. The algorithm for predictive parsing is given in Figure 4.6.

```

Repeat
  Set A to the top of the stack and a the next input symbol
  If (A is a terminal or $) then
    If (A = a) then
      pop A from the stack and remove a from the input
    else
      /* error occurred */
  else /* A is a non-terminal */
    if ( $M[A, a] = A \rightarrow B_1 B_2 B_3 \dots B_k$ ) then
      /*M is the parsing table for grammar G*/
      Begin
        Pop A from the stack
        Push  $B_k, B_{k-1}, \dots, B_1$  onto the stack,  $B_1$  as top
      End
  Until (A = $) /* stack becomes empty */

```

Figure 4.6 Algorithm of Predictive Parsing

In predictive parsing, a parsing table is constructed. To construct the table, we need two functions, namely, $FIRST()$ and $FOLLOW()$, that are associated with the grammar G . These two functions are used to fill the proper entries in the table for G , if such a parsing table for G exists. The algorithm to construct the predictive parsing table is given in Figure 4.7.

```

For each production  $X \rightarrow \alpha$  of grammar  $G$  do
  Begin
    For each terminal  $a$  in  $FIRST(\alpha)$ 
      add  $X \rightarrow \alpha$  to  $M[X, a]$ 
    End
  If ( $FIRST(\alpha)$  contains  $\epsilon$ ) then
    For each terminal  $b$  in  $FOLLOW(X)$  add  $X \rightarrow \alpha$  to  $M[X, b]$ 
  If ( $FIRST(\alpha)$  contains  $\epsilon$ ) And ( $FOLLOW(X)$  contains  $\$$ ) then

```

```

    add  $X \rightarrow \alpha$  to  $M[X, \$]$ 
if ( $M[X, a]$  contains no more productions) then
    Set  $M[X, a]$  to error

```

Figure 4.7 Algorithm to Construct Predictive Parsing Table

5. Explain the FIRST-FOLLOW procedure.

Ans: For a grammar G , we need two functions **FIRST** and **FOLLOW** to construct the predictive parsing table for G . The first and follow procedure permit us to select which production to apply, based on the next input symbol.

The $\text{FIRST}(A)$ is the set of all the terminals that A can begin with. If $A \xRightarrow{*} \epsilon$, then ϵ is also in $\text{FIRST}(A)$. To compute $\text{FIRST}(A)$ for all grammar symbols, the following rules can be applied:

1. If A is a terminal, then $\text{FIRST}(A) = \{A\}$.
2. If A is a non-terminal and $A \rightarrow X_1 X_2 \dots X_k \dots X_n$ is a production.
 - If $\text{FIRST}(X_1)$ does not contain ϵ , then $\text{FIRST}(A) = \text{FIRST}(X_1)$.
 - If $\text{FIRST}(X_1)$ contains ϵ and $\text{FIRST}(X_2)$ does not contain ϵ , then $\text{FIRST}(A) = \text{FIRST}(X_2)$.
 - If $\text{FIRST}(X_1)$ and $\text{FIRST}(X_2)$ both contain ϵ and $\text{FIRST}(X_3)$ does not contain ϵ , then $\text{FIRST}(A) = \text{FIRST}(X_3)$.

In general,

$\text{FIRST}(A) = \text{FIRST}(X_k)$ if $\text{FIRST}(X_1), \text{FIRST}(X_2) \dots \text{FIRST}(X_{k-1})$ all contain ϵ and $\text{FIRST}(X_k)$ does not contain ϵ .

3. If there exists a production $A \rightarrow \epsilon$, then add ϵ to $\text{FIRST}(A)$.

$\text{FOLLOW}(A)$ for a non-terminal A is the set of all the terminals that can follow A . The FOLLOW set does not contain ϵ . To compute $\text{FOLLOW}(A)$ for all grammar symbols A , these rules can be applied:

1. Set $\text{FOLLOW}(S) = \$$, where S is the start symbol and the symbol $\$$ indicates the end of input.
2. If there exists a production $A \rightarrow \beta X \gamma$, where γ is not ϵ , then everything in $\text{FIRST}(\gamma)$, except ϵ , is in $\text{FOLLOW}(X)$.
3. If there exists a production $A \rightarrow \beta X$, or $A \rightarrow \beta X \gamma$, where $\text{FIRST}(\gamma)$ contains ϵ , then all in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(X)$.

6. Define LL(1) grammars with an example.

Ans: The first 'L' in LL(1) stands for left to right scanning of input, the second 'L' to construct a leftmost derivation, and '1' indicates at each step, parser considers only one symbol of lookahead to make parsing decision. LL(1) grammars are very well-liked because the corresponding LL parsers only require to look at the next token to make their parsing decision. The LL(1) grammar is always non-left-recursive and unambiguous.

For a grammar G if $X \rightarrow \alpha \mid \beta$ are two dissimilar productions, then G is LL(1) if and only if the given conditions are satisfied:

- For no terminal b do both α and β derive strings that begin with b .
- Empty string can be derived by at most one of the production α and β .

- If $\beta \xRightarrow{*} \epsilon$, then α does not derive any string that starts with a terminal in $\text{FOLLOW}(X)$. Similarly, if $\alpha \xRightarrow{*} \epsilon$, then β does not derive any string that starts with a terminal in $\text{FOLLOW}(X)$.

For example, consider the following grammar to construct a LL (1) parse table:

$S \rightarrow aABb$

$A \rightarrow c | \epsilon$

$B \rightarrow d | \epsilon$

First, find out the FIRST and FOLLOW sets of all nonterminals.

$\text{FIRST}(S) = \{a\}$

$\text{FIRST}(A) = \{c, \epsilon\}$

$\text{FIRST}(B) = \{d, \epsilon\}$

$\text{FOLLOW}(S) = \{\$, \}$, since S is the start symbol.

$\text{FOLLOW}(A) = \text{FIRST}(Bb)$

$= \text{FIRST}(B) - \{\epsilon\} \cup \text{FIRST}(b)$

$= \{d, \epsilon\} - \{\epsilon\} \cup \{b\}$

$= \{d, b\}$

$\text{FOLLOW}(B) = \{b\}$

Now,

1. Considering the production $S \rightarrow aABb$

$\text{FIRST}(S) = \{a\}$

Since it does not contain any ϵ .

So, parse table $[S, a] = S \rightarrow aABb$

(1)

2. Considering the production $A \rightarrow c$

$\text{FIRST}(A) = \text{FIRST}(c) = \{c\}$.

Since it does not contain any ϵ .

So, parse table $[A, c] = A \rightarrow c$

(2)

3. Considering the production $A \rightarrow \epsilon$

$\text{FIRST}(A) = \text{FIRST}(\epsilon) = \{\epsilon\}$.

Since it contains ϵ . Thus, we have to find out $\text{FOLLOW}(A)$.

$\text{FOLLOW}(A) = \{d, b\}$

So, parse table $[A, d] = A \rightarrow \epsilon$

Also, parse table $[A, b] = A \rightarrow \epsilon$

(3)

4. Considering the production $B \rightarrow d$

$\text{FIRST}(B) = \text{FIRST}(d) = \{d\}$.

Since it does not contain any ϵ .

So, parse table $[B, d] = B \rightarrow d$

(4)

5. Considering the production $B \rightarrow \epsilon$

$\text{FIRST}(B) = \text{FIRST}(\epsilon) = \{\epsilon\}$.

Since it contains ϵ . Thus, we have to find out $\text{FOLLOW}(B)$.

$\text{FOLLOW}(B) = \{b\}$

So, parse table $[B, b] = B \rightarrow \epsilon$

(5)

Thus, the resultant parse table, from (1), (2), (3), (4), and (5) is shown in Table 4.1

Table 4.1 LL(1) Parsing Table

	a	b	c	d	\$
S	$S \rightarrow aABb$				
A		$A \rightarrow \epsilon$	$A \rightarrow c$	$A \rightarrow \epsilon$	
B		$B \rightarrow \epsilon$		$B \rightarrow d$	

7. Write down the algorithm for recursive-decent parsing. Explain with an example.

Ans: A recursive-decent parser is a collection of procedures one for each non-terminal. Starting from the start symbol, the parser continues its scanning until it stops and announces success if it scans entire input string. The algorithm for recursive-decent parsing is shown in Figure 4.8.

```

void X()
Begin
    Select an X-production,  $X \rightarrow A_1 A_2 \dots A_n$ 
    For (i = 1 to n) do
        Begin
            If ( $A_i$  is a non-terminal)
                call procedure  $A_i()$ 
            else if ( $A_i = a$ ) /* a is the current input symbol */
                lead the input to the next symbol
            else /* presence of some error */
        End
    End

```

Figure 4.8 Algorithm of Recursive-Decent Parsing

For example, consider the following grammar:

$X \rightarrow aMb$
 $M \rightarrow cd \mid c$

Input string is acb.

For constructing parse tree, starting from X, we check the available productions for X. As X has only one production, so we use it to spread X and obtain the tree shown in Figure 4.9(a). As the leftmost symbol matches with the first symbol of input string, we move toward the second symbol, which is M, so we expand M with first substitute $M \rightarrow cd$ to obtain the tree shown in Figure 4.9(b).

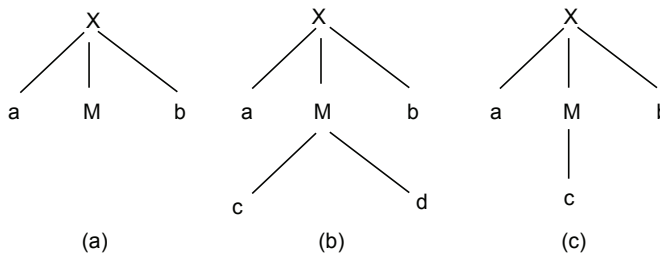


Figure 4.9 Parse Tree for Input String acb

Now, we get a match for the second symbol c , but as we proceed toward the third symbol a failure occurs because symbol b does not match with d . Now we go back to M and select second production of $M \rightarrow c$, and prepare its parse tree shown in Figure 4.9(c). The leaf c matches with the second symbol of input string and leaf b with the third symbol of input string. Hence, we declare a successful parsing for the input string.

8. Explain the non-recursive predictive parsing or table-driven predictive parsing.

Ans: The non-recursive predictive parser explicitly maintains a stack instead of implicit stack that is maintained via recursive calls. The parser imitates a leftmost derivation. For an input a , the stack holds a sequence of grammar symbols β such that $S \xRightarrow{lm} a\beta$.

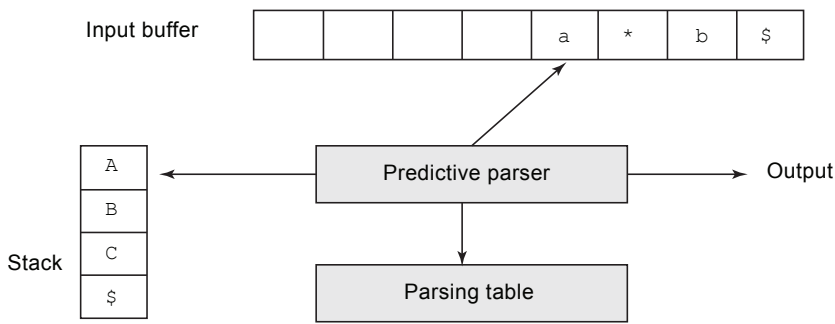


Figure 4.10 Non-recursive Predictive Parser

The non-recursive predictive parser contains:

- ❑ A stack containing a sequence of grammar symbol with $\$$ at the bottom of the stack,
- ❑ An input buffer containing string to be parsed with an end marker $\$$,
- ❑ A parsing table,
- ❑ An output stream.

The non-recursive predictive parser is controlled by a program that takes the top of the stack A , and the current input symbol a in consideration. If A is a non-terminal, then the parser chooses an A -production from parsing table M if there is an entry for $M[A, a]$. If A is a terminal, then it tries for a match between present input symbol a and the terminal A .

For a grammar G with a string d and a parsing table M , a leftmost derivation of d should exist if d is in $L(G)$. Let us consider $d\$$ in the input buffer and grammar start symbol S above $\$$ is on the top of stack. So the predictive parser uses its parsing table to parse the input according to the following procedure shown in Figure 4.11.

```
Set instruction pointer to point the first symbol of d
Set top stack symbol to A
```

```
While (A ≠ $) do
```

```
Begin
```

```
    If (A = a) then
```

```
        perform pop action and move ahead the instruction pointer to
        point to next symbol
```

```
    else if (A is a terminal) then
```



```

    /*error occurred*/
else if (M[A, a] is an error entry ) then
    /*error occurred*/
else if (M [A, a] =  $A \rightarrow B_1 B_2 B_3 \dots B_k$  then
Begin
    output the production  $A \rightarrow B_1 B_2 B_3 \dots B_k$ 
    pop the stack
    push  $B_k, B_{k-1}, \dots, B_1$  onto the stack,  $B_1$  on the top
End
set A to the top of stack
End

```

Figure 4.11 Non-recursive Predictive Parsing Algorithm

9. What are the advantages and disadvantages of table-driven predictive parsing?

Ans: Advantages:

- ❑ A table-driven parser can be easily generated from a given grammar. The parsing program is independent of the grammar but the parsing table depends on the grammar. With the use of FIRST and FOLLOW generation algorithms parsing table can be generated.
- ❑ Some entries in the parsing table have entries that point to the error recovery and reporting routines, which makes error recovery and reporting an easier task.

Disadvantage:

- ❑ Such type of parsers can work only on LL(1) grammars. Sometimes elimination of left-factoring and left-recursion may not be sufficient to transform a grammar into LL(1) grammar.

10. Explain the error recovery strategies in predictive parsing.

Ans: A top-down predictive parser can be implemented by recursive-decent parsing or by table-driven parsing. The table-driven parsing predicts as to what terminals and non-terminals the parser expects from the rest of the input. An error can occur in the following situations:

- ❑ If the terminal on the top of a stack does not match the next input symbol.
- ❑ If a non-terminal A is on the top of stack, x is the next input symbol and the parsing table entry M [A, x] is empty.

Two commonly used error recovery schemes, namely, *panic mode recovery* and *phrase level recovery*.

Panic mode recovery is based on the idea that when an error occurs, the parser skips the input symbols until it finds a **synchronizing token** (a semicolon, }, or any other token with an unambiguous and clear role) in the input. A set of all synchronizing tokens is known as **synchronizing set**. The synchronizing set should be enough effective to recover from errors and it depends on the choice of synchronizing set. Some guidelines for constructing a synchronizing set are as follows:

- ❑ For a non-terminal A, place all the elements of FOLLOW (A) into the synchronizing set of A. Skip the tokens until an element of FOLLOW (A) is found and then pop A from the stack.
- ❑ For a non-terminal A, all the elements of FIRST (A) can also be added to the synchronizing set of A. This will help the parser in resuming the parsing according to A, if a symbol in FIRST (A) appears in the input.
- ❑ The production that derives ϵ can be employed as a default production, if a non-terminal can produce the empty string. It may delay the error detection for some time but cannot cause an error to

be missed during error recovery. This approach is useful in reducing the number of non-terminals to be parsed.

- A terminal on the top of stack, which cannot be matched, is popped from the top of stack and a warning message indicating that the terminal was inserted is issued. After issuing the warning message, the parser can continue parsing as if the missing symbol is a part of the input. In effect, this approach takes all other tokens into the synchronizing set for a token.

Phrase level recovery is based on the idea of filling the blank entries in the predictive parsing table with pointers to error handling routines. The error handling routines can do the following:

- They can insert, modify, or delete any symbols in the input.
- They can also issue appropriate error messages.
- They can pop elements from the stack.

Pushing a new symbol onto the stack and alteration of the existing symbol in stack is problematic due to some reasons as given below:

- The steps performed by the parser may result in the derivation of a word that does not correspond to the derivation of any word in the language at all.
- There is a possibility of infinite loop formation during the alteration of stack symbol.

11. Explain bottom-up parsing with an example. Also discuss reduction in bottom-up parsing

Ans: Bottom-up parsing is a parsing method to construct a parse tree for an input string beginning at the leaves and growing towards the root. Bottom-up parsing can be considered as a process of reducing the input string to the start symbol of the grammar. At each reduction step, a particular substring matching the right hand side of a production is replaced by the non-terminal symbol on the left hand side of the production. Bottom-up parser can handle a large class of grammar.

For example, the steps to construct a parse tree of the token stream $a + b$ with respect to a grammar $G, S \rightarrow S * S \mid S + S \mid S - S \mid a \mid b$ are shown in Figure 4.12.

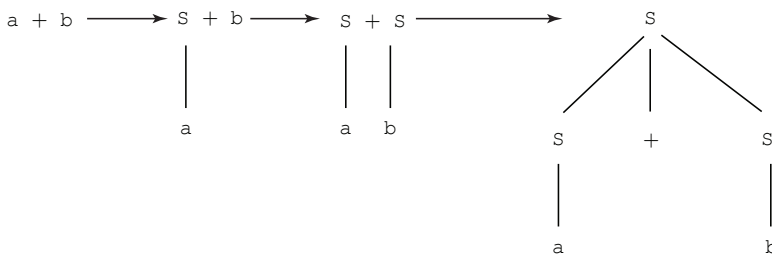


Figure 4.12 Steps to Construct a Bottom-up Parse Tree for $a + b$

The sequence starts with the input string $a + b$. The first reduction generates $S + b$ by reducing a to S , using the production $S \rightarrow a$. The second reduction generates $S + S$ by reducing b to S , using the production $S \rightarrow b$. Finally, the reduction of $S + S$ to the start symbol S is done using the production $S \rightarrow S + S$.

12. Define handle. Write a short note on handle pruning.

Ans: Handle of a string is a substring that matches the right hand side of some production and whose reduction to the non-terminal on the left hand side is a step along the reverse of some rightmost derivation. In other words, handle of a right sentential form γ is $\langle A \rightarrow \beta \rangle$, the location of β in γ , such that replacing β by A at that position generates the previous right sentential form in a rightmost derivation of γ .

Formally, if

$$S \xRightarrow[\text{rm}]{*} \alpha A \gamma \Rightarrow \alpha \beta \gamma$$

Then, $A \rightarrow \beta$ is a handle of $\alpha \beta \gamma$ at the location immediately after α .

Note that a certain sentential form may have many different handles but the right sentential forms of a non-ambiguous grammar have unique handle.

Consider the following productions to drive a string `abbcede`:

$S \rightarrow aABe$
 $A \rightarrow Abc \mid b$
 $B \rightarrow d$

The rightmost derivation is as follows:

$$S \Rightarrow \underline{aABe} \Rightarrow aA\underline{de} \Rightarrow aA\underline{bc}de \Rightarrow \underline{ab}bcde$$

It follows that

$S \rightarrow aABe$ is a handle of aABe in location 1.
 $B \rightarrow d$ is a handle of aAde in location 3.
 $A \rightarrow Abc$ is a handle of aAbcde in location 2.
 $A \rightarrow b$ is a handle of abbcde in location 2.

Handle pruning: The process of discovering a handle and reducing it to the suitable left hand side is called handle pruning. A rightmost derivation in reverse (termed as **canonical reduction sequence**) is obtained by handle pruning. Handle pruning forms the basis of bottom-up parsing. Starting from a terminal string w , where w is a sentence of the grammar, let $w = \alpha_n$, where α_n is the n^{th} right sentential form of some as yet unknown rightmost derivation.

To construct a rightmost derivation

$$S \Rightarrow \alpha_0 \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_{n-1} \Rightarrow \alpha_n \Rightarrow w$$

We can apply the following algorithm:

For $i \leftarrow n$ to 1 by -1
 Find the handle $\langle A_i \rightarrow \beta_i, k_i \rangle$ in α_i
 Replace β_i with A_i to generate α_{i-1}

This algorithm requires $2*n$ steps.

For example, the sequence of reductions to reduce `abbcede` to start symbol S is given in Table 4.2.

Table 4.2 Reduction of `abbcede` to S

Right Sentential Form	Handle	Reducing Production
<u>ab</u> bcde	b	$A \rightarrow b$
a <u>Ab</u> cde	Abc	$A \rightarrow Abc$
aA <u>d</u> e	d	$B \rightarrow d$
<u>aABe</u>	aABe	$S \rightarrow aABe$
S		

13. Explain shift-reduce parsing with stack implementation.

Ans: Shift-reduce parsing is a kind of bottom-up parsing in which a stack is used to hold the grammar symbols and an input buffer is used to hold the remaining string to be parsed. The parser examines the input tokens and either shift (push) them onto a stack or reduce symbols at the top of the stack, replacing a right hand side by a left hand side. Though only shift and reduce symbols are considered as major operations but in fact a shift-reduce parser can make four actions:

- ❑ **Shift:** A shift action corresponds to pushing the next input symbol onto the top of stack.
- ❑ **Reduce:** A reduce action occurs when we have the right end of the handle at the top of the stack. To perform reduction, we locate the left end of the handle within the stack and choose a non-terminal on the left hand side of the corresponding rule to replace the handle.
- ❑ **Accept:** An accept action occurs when parser declares the successful completion of parsing.
- ❑ **Error:** An error action occurs when the parser finds a syntax error in the input and then parser calls an error recovery routine.

The symbol \$ is used to mark the bottom of the stack and the right end of the input string. Initially, the stack is empty and the input string is on the input buffer as shown below:

<u>Stack</u>	<u>Input Buffer</u>
\$	w\$

The parser performs a left-to-right scan through the input string to shift zero or more symbols onto the stack until it locates a prefix of the symbol (handle) on the top of the stack that matches the right hand side of a grammar rule. Then, the parser reduces the right hand side symbols on the top of the stack with the non-terminal occurring on the left hand side of the grammar rule. The parser repeats the process until it reports an error or a success message. The parsing is said to be successful, if the stack contains the start symbol and the input is empty as shown below:

<u>Stack</u>	<u>Input Buffer</u>
\$S	\$

Consider the following grammar:

$$S \rightarrow S + S \mid S * S \mid (S) \mid a$$

To parse the input string $a + a$, a shift-reduce parser performs a sequence of steps as shown in Table 4.3.

Table 4.3 Shift-reduce Parsing Actions

Stack	Input Buffer	Action
\$	a + a\$	Shift
\$a	+a\$	Reduce $S \rightarrow a$
\$S	+a\$	Shift
\$S+	a\$	Shift
\$S + a	a\$	Reduce $s \rightarrow a$
\$S + S	\$	Reduce $S \rightarrow S + S$
\$S	\$	Accept

14. Explain operator precedence parsing method of shift-reduce parsing.

Ans: Operator precedence parsing is a shift-reduce parsing technique that can be applied to operator grammar. An **operator grammar** is a small, but an important class of grammars in which no production rule can have:

- ❑ An $\in$ production on the right hand side and
- ❑ Two adjacent non-terminals at the right hand side.

An operator precedence parser consists of the following:

- ❑ An input buffer containing the input string to be parsed,
- ❑ A stack containing the sequence of grammar symbols,
- ❑ An operator precedence relations table,
- ❑ A precedence parsing program,
- ❑ An output.

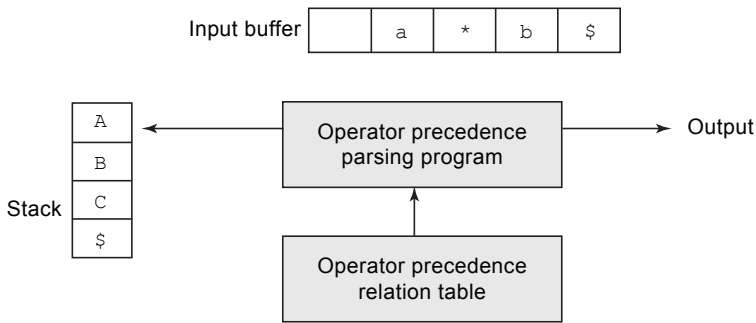


Figure 4.13 Operator Precedence Parser

There are three disjoint precedence relations that can exist between the pairs of terminals.

- ❑ $a < b$ b has higher precedence than a .
- ❑ $a \doteq b$ b has same precedence as a .
- ❑ $a > b$ b has lower precedence than a .

Table 4.4 Operator Precedence Relations

[illegible]

For example, consider the following grammar:

$$S \rightarrow S * S \mid S / S \mid S - S \mid id$$

Then the input string $id + id * id$ after inserting the precedence relations will be:

$$\$ < \cdot id \cdot > + < \cdot id \cdot > * < \cdot id \cdot > \$$$

The operator precedence parsing algorithm to generate a parse tree for the input string is shown in Figure 4.14.

Input: An input string $w\$$, a table holding precedence relations and the stack with initial symbol $\$$.

Output: Parse tree.

Algorithm:

```

Set p to point to the first symbol of w$
Repeat forever
  If ($ is on top of the stack and p points to $) then
    accept and exit the loop
  else
    Begin
      let terminal a is on the top of the stack and let
      b be the symbol pointed to by p
      If ( $a < \cdot b$  or  $a \doteq b$ ) then /* Shift */
        Begin
          push b onto the stack
          advance p to the next input symbol
        End
      else if ( $a \cdot > b$ ) then /* Reduce */
        Repeat
          pop stack
        Until (the top of stack terminal is related by  $< \cdot$ 
          to the terminal most recently popped);
      else /*error occurred*/
    End

```

Figure 4.14 Operator Precedence Algorithm

15. What are the advantages and disadvantages of operator precedence parsing?

Ans: Advantages:

- ❑ Operator precedence parsing is simple and easy to implement.
- ❑ Its parser is constructed by hand after knowing the grammar.
- ❑ Debugging is simple.

Disadvantages:

- ❑ Tokens like minus ($-$) are difficult to handle, as depending on whether it is being used as binary operator or unary operator it has two different values of precedence.
- ❑ It does take grammar as an input while generating a parser. This results in rewriting of the parser in case of any additions or deletions in the production rules, which is very cumbersome and time-consuming process.
- ❑ Only a small class of grammars like operator grammars can be parsed by this parsing technique.

16. Find FIRST of all the non-terminals of the following grammar:

$$S \rightarrow ACB | CbB | Ba$$

$$A \rightarrow da | BC$$

$$B \rightarrow g | \epsilon$$

$$C \rightarrow h | \epsilon$$

Ans:

$$1. \text{ FIRST}(C) = \text{FIRST}(h) \cup \text{FIRST}(\epsilon)$$

$$= \{h\} \cup \{\epsilon\}$$

$$= \{h, \epsilon\}$$

$$2. \text{ FIRST}(B) = \text{FIRST}(g) \cup \text{FIRST}(\epsilon)$$

$$= \{g\} \cup \{\epsilon\}$$

$$= \{g, \epsilon\}$$

$$3. \text{ FIRST}(A) = \text{FIRST}(da) \cup \text{FIRST}(BC)$$

$$= \{d\} \cup \{\text{FIRST}(B) - \{\epsilon\} \cup \text{FIRST}(C)\}$$

$$= \{d\} \cup \{\{g, \epsilon\} - \{\epsilon\} \cup \{h, \epsilon\}\}$$

$$= \{d\} \cup \{g\} \cup \{h, \epsilon\} \quad [\text{from (1) and (2)}]$$

$$= \{d\} \cup \{g, h, \epsilon\}$$

$$= \{d, g, h, \epsilon\}$$

$$4. \text{ FIRST}(S) = \text{FIRST}(ACB) \cup \text{FIRST}(CbB) \cup \text{FIRST}(Ba)$$

We first determine $\text{FIRST}(ACB)$, $\text{FIRST}(CbB)$, and $\text{FIRST}(Ba)$ separately,

$$\text{FIRST}(ACB) = \text{FIRST}(A) - \{\epsilon\} \cup \text{FIRST}(BC)$$

$$= \{d, g, h, \epsilon\} - \{\epsilon\} \cup \text{First}(B) - \{\epsilon\} \cup \text{First}(C)$$

$$= \{d, g, h\} \cup \{g, \epsilon\} - \{\epsilon\} \cup \{h, \epsilon\}$$

$$= \{d, g, h\} \cup \{g\} \cup \{h, \epsilon\}$$

$$= \{d, g, h, \epsilon\}$$

$$\text{FIRST}(CbB) = \text{FIRST}(C) - \{\epsilon\} \cup \text{FIRST}(bB)$$

$$= \{h, \epsilon\} - \{\epsilon\} \cup \{b\}$$

$$= \{h\} \cup \{b\}$$

$$= \{h, b\}$$

$$\text{FIRST}(Ba) = \text{FIRST}(B) - \{\epsilon\} \cup \text{FIRST}(a)$$

$$= \{g, \epsilon\} - \{\epsilon\} \cup \{a\}$$

$$= \{g\} \cup \{a\}$$

$$= \{g, a\}$$

Finally, by combining the value of $\text{FIRST}(ACB)$, $\text{FIRST}(CbB)$, and $\text{FIRST}(Ba)$, we get

$$\text{FIRST}(S) = \{d, g, h, \epsilon\} \cup \{h, b\} \cup \{g, a\}$$

$$= \{d, g, h, b, a, \epsilon\}$$

17. Consider the following grammar and show the handle of each right sentential form for the string (b, (b, b)).

$$E \rightarrow (A) | b$$

$$A \rightarrow A, E | E$$

Ans: The following sentential form will occur in reduction of $(b, (b, b))$ to S .

1. $(b, (b, b))$ (first b is the handle)
2. $(E, (b, b))$ (E is the handle)
3. $(A, (b, b))$ (first b is the handle)
4. $(A, (E, b))$ (E is the handle again)
5. $(A, (A, b))$ (b is the handle)
6. $(A, (A, E))$ ((A, E) is the handle)
7. $(A, (A))$ ((A) is the handle)
8. (A, E) (again (A, E) is the handle)
9. (A) ((A) is the handle)
10. E (finally string is reduced to starting non-terminal)

18. Consider the following grammar:

$S \rightarrow SAS$
 $S \rightarrow \text{num}$
 $A \rightarrow +$
 $A \rightarrow -$
 $A \rightarrow *$
 $A \rightarrow /$

Explain why this grammar is not suitable to form the basis for a recursive-decent parser. Use left-factoring and left-recursion removal to obtain an equivalent grammar which can be used as the basis for recursive-decent parser.

Ans: Consider the following production:

$S \rightarrow SAS$

If we put the value of S in place of first S at the right hand side in this production, the new production will be

$S \rightarrow SASAS$

If we again put the value of S in place of first S at the right hand side, the new production will be

$S \rightarrow SASASAS$

Thus, putting the value of S in place of first S at the right hand side again and again will result in an infinite loop. It shows that the given grammar suffers from the problem of left recursion. Hence, it cannot be the basis for recursive-decent parser.

If we put the value of A in the above production, we get

$S \rightarrow S + S$
 $S \rightarrow S - S$
 $S \rightarrow S * S$
 $S \rightarrow S / S$
 $S \rightarrow \text{num}$

It results in the following production:

$S \rightarrow S + S \mid S - S \mid S * S \mid S / S \mid \text{num}$

It still suffers from left recursion, which can be removed by following the algorithm discussed in Chapter 3. Now, we have the following productions:

$$\begin{aligned} S &\rightarrow \text{num } S' \\ S &\rightarrow +S S' \mid \in \\ S &\rightarrow -S S' \mid \in \\ S &\rightarrow *S S' \mid \in \\ S &\rightarrow /S S' \mid \in \end{aligned}$$

This grammar does not suffer from left recursion, and hence, can form the basis for recursive-decent parser. The production will now become:

$$S \rightarrow \text{num } S' \mid +S S' \mid -S S' \mid *S S' \mid /S S' \mid \in \mid \text{num}$$

19. Show that the given grammar is not LL(1).

$$\begin{aligned} E &\rightarrow iAcE \mid iAcEeE \mid a \\ A &\rightarrow b \end{aligned}$$

Ans: Step 1: This grammar suffers from left factoring, so after removing left factoring

$$\begin{aligned} E &\rightarrow iAcEE' \mid a \\ E' &\rightarrow eE \mid \in \\ A &\rightarrow b \end{aligned}$$

Step 2: Compute FIRST and FOLLOW of all non-terminals.

$$\begin{aligned} \text{FIRST } (E) &= \{i, a\} \\ \text{FIRST } (E') &= \{e, \in\} \\ \text{FIRST } (A) &= \{b\} \\ \text{FOLLOW } (E) &= \{\$, e\} \\ \text{FOLLOW } (E') &= \{\$, e\} \\ \text{FOLLOW } (A) &= \{c\} \end{aligned}$$

Now, to generate the parser table entries, follow these steps:

1. Considering the production $E \rightarrow iAcEE'$
 $\text{FIRST}(E) = \text{FIRST}(iAcEE') = \{i\}$
 Since it does not contain any $\in$.
 So, parse table $[E, i] = E \rightarrow iAcEE'$
2. Considering the production $E \rightarrow a$
 $\text{FIRST}(E) = \text{FIRST}(a) = \{a\}$
 Since it does not contain any $\in$.
 So, parse table $[E, a] = E \rightarrow a$
3. Considering the production $E' \rightarrow eE$
 $\text{FIRST}(E') = \text{FIRST}(eE) = \{e\}$
 Since it does not contain any $\in$.
 So, parse table $[E', e] = E' \rightarrow eE$
4. Considering the production $E' \rightarrow \in$
 $\text{FIRST}(E') = \text{FIRST}(\in) = \{\in\}$
 Since it contains an $\in$, we have to find out FOLLOW (E') .
 $\text{FOLLOW}(E') = \{\$, e\}$

So, parse table $[E', \$] = E' \rightarrow \epsilon$

Also, parse table $[E', e] = E' \rightarrow \epsilon$

5. Considering the production $A \rightarrow b$

$FIRST(A) = FIRST(b) = \{b\}$

Since it does not contain any ϵ .

So, parse table $[A, b] = A \rightarrow b$

The resultant parse table is shown in Table 4.5.

Table 4.5 LL(1) Parsing Table

	a	b	e	i	c	\$
E	$E \rightarrow a$			$E \rightarrow iAcEE'$		
E'			$E' \rightarrow eE$ $E' \rightarrow \epsilon$			$E' \rightarrow \epsilon$
A		$A \rightarrow b$				

Multiple entries

The multiple entries in $M[E', e]$ field show that the grammar is ambiguous and not LL(1).

Multiple-Choice Questions

- Top-down parsing is a technique to find _____.
 - Leftmost derivation
 - Rightmost derivation
 - Leftmost derivation in reverse
 - Rightmost derivation in reverse
- Predictive parsing is possible only for _____.
 - LR(k) grammar
 - LALR(1) grammar
 - LL(k) grammar
 - CLR(1) grammar
- Which two functions are required to construct a parsing table in predictive parsing technique?
 - CLOSURE() and GOTO()
 - FIRST() and FOLLOW()
 - ACTION() and GOTO()
 - None of these
- Non-recursive predictive parser contains _____.
 - An input buffer
 - A parsing table
 - An output stream
 - All of these
- Which of these parsing techniques is a kind of bottom-up parsing?
 - Shift-reduce parsing
 - Reduce-reduce parsing
 - Predictive parsing
 - Recursive-decent parsing
- Which of the following methods is used by the bottom-up parser to generate a parse tree?
 - Leftmost derivation
 - Rightmost derivation
 - Leftmost derivation in reverse
 - Rightmost derivation in reverse
- Handle pruning forms the basis of _____.
 - Bottom-up parsing
 - Top-down parsing
 - Both (a) and (b)
 - None of these

8. In shift-reduce parsing, accept action occurs _____.
 (a) When we have the right end of the handle at the top of the stack
 (b) When we have the left end of the handle at the top of the stack
 (c) When parser declares the successful completion of parsing
 (d) When the parser finds a syntax error in the input and calls an error recovery routine
9. Which of the following operators is hard to handle by the operator precedence parser?
 (a) Plus (+) (b) Minus (−)
 (c) Multiply (*) (d) Divide (/)
10. Given a grammar G:
 $T \rightarrow BCTd \mid Bcd$
 $CB \rightarrow BC$
 $Cc \rightarrow cc$
 $Bc \rightarrow bc$
 $Bb \rightarrow b$

Which of the following sentences can be derived by G?

- (a) bcd (b) bbc
 (c) bcdd (d) bccd

Answers

1. (a) 2. (c) 3. (b) 4. (d) 5. (a) 6. (d) 7. (a) 8. (c) 9. (b) 10. (a)

LR Parsers

1. Explain LR parsers. What are its components?

Ans: **LR parsers** are efficient bottom-up parsers for a large class of context-free grammars. An LR parser is a non-backtracking shift-reduce parser in which 'L' indicates that they scan input from left to right and 'R' indicates that they construct a rightmost derivation in reverse. **LR parsing** is a method for syntactic recognition of programming languages. It makes use of tables to determine when a rule is complete and which additional tokens must be read from the source string. The term LR(k) can also be used to represent LR parser, where k indicates the number of input symbols of lookahead that are used in making parsing decision. Only the cases where the value of k is either 0 or 1 are of practical interest. If the value of k is not defined, it is taken as 1. So an LR parsing can be considered as an LR(1) parsing, that is, LR parsing with one symbol of lookahead.

Logically, an LR parser consists of two parts, a *driver routine* and a *parsing table*. The driver routine is same for all LR Parsers, only the parsing table changes from one parser to another. There are three major methods for constructing LR parsing table.

- ❑ **Simple LR or SLR parsing:** It is easy to implement but less powerful than other parsing methods.
- ❑ **Canonical LR or LR(1) parsing:** It is most general and powerful, but is tedious and costly to implement. It contains more number of states as compared to the SLR.
- ❑ **Lookahead LR or LALR(1) parsing:** It lies in between SLR and canonical LR in terms of power; however, it can be implemented efficient with little bit of effort. It contains the same number of states as the SLR.

The LR parser is a state machine. The architecture of an LR parser is shown in Figure 5.1. It consists of the following:

- ❑ an input buffer,
- ❑ a stack, containing a list of states $s_0 s_1 \dots s_m$, where s_m is on top,
- ❑ a goto table that suggests to which state it should move,
- ❑ an action table that provides a grammar rule to be applied,
- ❑ a set of CFL (context-free language) rules.

The LR parsing program uses the combination of the stack symbol on the top of the stack and the current input symbol to index the parsing table. The parsing table consists of two sub tables: ACTION table and GOTO table.

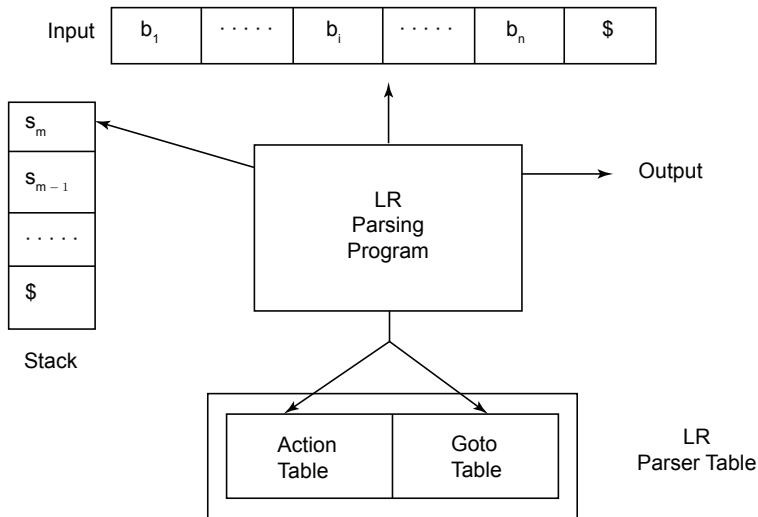


Figure 5.1 LR Parser

2. Why LR parsing is good and attractive? Also explain its demerits, if any.

Ans: LR parsing method is good and attractive due to the following reasons:

- ❑ LR parsing is the most common non-backtracking shift-reduce parsing.
- ❑ It is possible to construct the LR parsers for recognition of almost all programming language constructs for which CFG can be written.
- ❑ The class of grammars that can be parsed with predictive parsers can also be parsed using LR parsers. That is, the class of grammars that can be parsed with predictive parsers is a proper subset of those that can be parsed using LR parsers.
- ❑ An LR parser scans the input from left to right and while scanning it can detect the syntax errors as quickly as possible.

The main drawback of LR parsing is that for complex programming language grammars, the construction of LR parsing tables requires too much manual work. To reduce this manual work, we require an automated tool, known as **LR parser generator** that can generate an LR parser from a given grammar. Some available generators are YACC, bison, etc. These generators take context-free grammars as input and generate a parser for the input CFG. These generators also help in locating errors in the grammar, if any and generate error messages.

3. Explain ACTION and GOTO function in LR parsing.

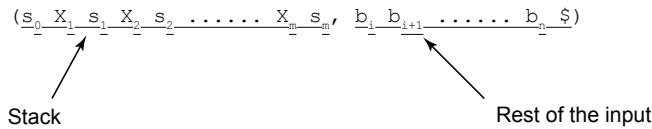
Ans: While constructing a parsing table, we consider two types of functions: a parsing-action function **ACTION** and a goto function **GOTO**.

- ❑ **ACTION function:** The **ACTION** function takes a state s_m (the state on the top of stack) and a terminal b_i (the current input symbol) as input to take an action. The $\text{ACTION}[s_m, b_i]$ can have one of the four values:
 - **Shift s :** The action of the parser is to shift input b to the stack. Here, the parser uses state s to represent b .
 - **Reduce $x \rightarrow \alpha$:** The action of the parser is to reduce α on the top of the stack to head x .

- **Accept:** The parser accepts the input and announces successful parsing for the input string.
 - **Error:** The parser finds an error and calls an error handling routine.
- **GOTO function:** The function GOTO can be defined as a set of states that takes a state and a grammar symbol as arguments and produces a new state. If $\text{GOTO}[s_i, B] = s_j$, then GOTO maps a state s_i and a non-terminal B to state s_j .

4. Explain configurations in LR parsing.

Ans: An LR parser configuration is a combination of two components. The first component is the stack content, which is a string of states and grammar symbols in the form $s_0 x_1 s_1 x_2 s_2 \dots x_m s_m$, and the second component is the remaining input (the input which is still unexpanded). The state s_0 is the start state of the parser that does not represent a grammar symbol. It mainly serves as bottom-of-stack marker. The configuration of an LR parser can be represented as follows:



The combination of s_m (the state symbol on the top of the stack) and b_i (the current input symbol) decides the parser action by consulting the parsing action table. Initial stack contains only s_0 . A configuration of an LR parsing represents the right sentential form

$$x_1 \dots x_m, b_i b_{i+1} \dots b_n \$$$

in the same way as that of shift-reduce parser. The only difference is that instead of grammar symbols, the stack contains those states from which grammar symbols can be recovered. That is, the grammar symbol x_j in right sentential form corresponds to state s_i in the configuration.

The configuration resulting after each of the four types of move is as follows:

- If $\text{ACTION}[s_m, b_i] = \text{shift } s$, the parser performs shift operation, that is, it shifts the s state (next state) onto the stack. The configuration now becomes:

$$(s_0 s_1 \dots s_m s, b_{i+1} \dots b_n \$)$$

Note that the current input symbol is b_{i+1} and there is no need to hold the symbol b_i on the stack, as it can be recovered from S if required.

- If $\text{ACTION}[s_m, b_i] = \text{reduce } X \rightarrow \alpha$, the parser performs a reduce operation. The new configuration is:

$$(s_0 s_1 \dots s_{m-p} s, b_i b_{i+1} \dots b_n \$)$$

where

p is the length of α (the body of the reducing production)

$$s = \text{GOTO}[s_{m-p}, X]$$

The parser first pops the p state symbols from the stack, which exposes the state s_{m-p} . Then, it pushes the state s , which is the entry for $\text{GOTO}[s_{m-p}, X]$, onto the stack. Note that b_i is still the current input symbol, that is, the reduce operation does not alter the current input symbol.

- If $\text{ACTION}[s_m, b_i] = \text{accept}$, it indicates the completion of parsing and the string is accepted.
- If $\text{ACTION}[s_m, b_i] = \text{error}$, an error is encountered by the parser and an error recovery routine is called.

5. Write the LR parsing algorithm.

Ans: Let us consider an input string w and an LR parsing table for grammar G with ACTION and GOTO functions. The algorithm for LR parsing is given in Figure 5.2.

```

Set instruction pointer to point to first symbol  $b$  of  $w\$$ 
Do
    Set  $s$  as the state on top of the stack
    If (ACTION[ $s, b$ ] = shift  $p$ ) then
        push  $p$  onto the stack and move ahead instruction
        pointer to point to next symbol
    else if (ACTION[ $s, b$ ] = reduce  $X \rightarrow \alpha$ ) then
        Begin
            pop  $|\alpha|$  symbols from the stack
            let  $p$  is now on the top of stack
            push GOTO[ $p, X$ ] onto the stack
            output the reducing production  $X \rightarrow \alpha$ 
        End
    else if (ACTION[ $s, b$ ] = accept) then
        exit the loop /* parsing is completed */
    else
        /* error occurred */
While (1)

```

Figure 5.2 LR Parsing Algorithm

6. Define LR(0) items and LR(0) automaton. Also explain the canonical LR(0) collection of items.

Ans: An **LR(0) item** (in short **item**) is defined as a production of grammar G having a dot at some position of the right hand side of the production. Formally, an item describes how much of a production has already been seen on the input at some point in the parsing process. For example, consider the following four items created by the production $X \rightarrow ABC$:

- $X \rightarrow .ABC$, which indicates that a string derivable from ABC is expected next on the input.
- $X \rightarrow A.BC$, which indicates that a string derivable from A has already been seen on the input and now a string derivable from BC is expected.
- $X \rightarrow AB.C$, which indicates that a string derivable from AB has already been seen and now a string derivable from C is expected on the input.
- $X \rightarrow ABC.$, which indicates that the body of the production has already been seen, and now it is time to reduce it to X .

The **canonical LR(0) collection** is a collection of sets of LR(0) items, which provides the basis for constructing a DFA that is used to make parsing decisions. Such an automaton is called an **LR(0) automaton**. The states in the LR(0) automaton correspond to the sets of items in the canonical LR(0) collection. The canonical LR(0) collection for a grammar can be constructed by defining its augmented grammar and two functions, CLOSURE and GOTO.

For a grammar G with start symbol S , G' will be the augmented grammar of G , with a new start symbol S' and production $S' \rightarrow S$. This new generated production indicates that when the parser

should stop parsing and announce the acceptance of the input. Thus, we can say that acceptance occurs only when the parser is about to reduce by $S' \rightarrow S$.

□ **CLOSURE**: Let I be the set of items for a grammar G , then we construct a set of items $\text{CLOSURE}(I)$ from I by following these steps:

- Add every item in I to $\text{CLOSURE}(I)$.
- If $A \rightarrow \alpha.B\beta$ is in $\text{CLOSURE}(I)$, where B is a non-terminal and $B \rightarrow \gamma$ is a production in G , then add the item $B \rightarrow \gamma$ to $\text{CLOSURE}(I)$, if it is not already present in it.
- Repeat step 2 until there are no more items to be added in $\text{CLOSURE}(I)$.

In step 2, $A \rightarrow \alpha.B\beta$ in $\text{CLOSURE}(I)$ represents that a substring derivable from $B\beta$ is expected to be seen as input at some point in the parsing process. The substring derivable from $B\beta$ will have the prefix derivable from B by applying one of the productions of B . Thus, we include items for all productions of B in $\text{CLOSURE}(I)$. For this reason, we include $B \rightarrow \gamma$ in $\text{CLOSURE}(I)$.

□ **GOTO**: If I is a set of items, X is a grammar symbol, and an item $(A \rightarrow \alpha.X\beta)$ is in I , then the function $\text{GOTO}(I, X)$ is defined as the closure of the set of all items $(A \rightarrow \alpha X\beta)$. The **GOTO** function basically defines the transitions in the LR(0) automaton. The states in LR(0) automaton are represented as set of items, and $\text{GOTO}(I, X)$ defines the transition from state for I for the given input X .

7. What is a Simple LR parser or SLR parser?

Ans: **SLR parser** is the simplest LR parser technique generating the parsing tables like LR(0) Parser. But unlike LR(0) parser, it only performs a reduction with a grammar rule $A \rightarrow w$ if the next input symbol is in $\text{FOLLOW}(A)$. This parser can prevent shift-reduce and reduce-reduce conflicts occurring in LR(0) parsers. Therefore, it can deal with more grammars. A grammar that can be parsed by an SLR parser is called an **SLR grammar**. For example, a grammar that can be parsed by SLR parser but not by an LR(0) is given below:

$$E \rightarrow 1 | E$$

$$E \rightarrow 1$$

8. Explain how to construct an LR(0) parser.

Ans: The various steps for constructing an LR(0) parser are given below:

1. For a grammar G with a start symbol S , construct an augmented grammar G' with a new start symbol S' and production $S' \rightarrow S$.
2. Compute the canonical collection of LR(0) items of grammar G .
3. Find the state transitions for state i for all non-terminals X using the following **GOTO** function:

$$I_1 = \text{GOTO}(I_0, X).$$

4. Construct new states with the help of $\text{CLOSURE}(I)$ and $\text{GOTO}(I, X)$ functions, and for each state construct new LR(0) items.
5. Repeat step 4 until there are no more transitions left on input for which state transitions can be constructed.
6. With all computed states $I_0, I_1, \dots, I_n$, construct the transition graph by keeping LR(0) items of each I_i in single node and linking these nodes with suitable transitions evaluated with **GOTO**.
7. Constitute a parse table using SLR table construction algorithm.
8. Apply the shift-reduce action to verify whether the input string is accepted or any conflict has occurred.

9. Discuss the algorithm to construct an SLR parsing table.

Or

How to make an ACTION and GOTO entry in SLR parsing table?

Ans: SLR parsing tables are constructed using two functions, namely, ACTION and GOTO. It begins with LR(0) items and LR(0) automaton, that is, for a grammar G , an augmented grammar G' with a new start symbol S' is constructed. For this augmented grammar G' , the canonical collection of items C is computed with the help of GOTO function. And then we construct the ACTION and GOTO entries in the parsing table using the following algorithm:

Step 1: For any augmented grammar G' , construct the collection of sets of LR(0) items $C = \{I_0, I_1, \dots, I_n\}$.

Step 2: For each state in C , construct a row in SLR table and name the rows from 0 to n . Partition the columns into ACTION and GOTO, where ACTION will have all terminal symbols of grammar G along with symbol $\$,$ and GOTO will have all the non-terminals of G .

Step 3: For each I_i construct a state i . The action entries for state i in SLR parsing table are determined using the following rules:

- ❑ If $[A \rightarrow \alpha.b\beta]$ is in I_i and $\text{GOTO}(I_i, b) = I_j$, where b is a terminal, then $\text{ACTION}[i, b] = \text{"shift } j\text{"}$, which implies an entry corresponding to state i and terminal j is made in the ACTION part.
- ❑ For all b in $\text{FOLLOW}(A)$, if $[A \rightarrow \alpha.]$ is in I_i , then $\text{ACTION}[i, b] = \text{"reduce } A \rightarrow \alpha\text{"}$.
- ❑ If $[S' \rightarrow S.]$ is in I_i , then $\text{ACTION}[i, \$] = \text{"accept"}$.

If any conflicting actions occur from these rules, we will not consider the grammar as SLR(1). In that case, this algorithm will not be able to produce a valid parser.

Step 4: The goto entries for state i in the SLR parsing table can be determined using the following rule:

If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}[i, A] = j$.

Step 5: The undefined entries are made 'error' entries.

Step 6: The initial state of parser is one constructed from the set of items containing $[S' \rightarrow .S]$.

The parsing table containing ACTION and GOTO functions defined by the above algorithm is known as **SLR(1) parsing table** for G , and the LR parser that makes use of this table is known as **SLR(1) parser** for G . Here, 1 in SLR(1) denotes the single lookahead symbol.

10. What are the demerits of SLR parser?

Ans: The SLR parser has certain demerits given below:

- ❑ A state may include a final item and a non-final item. This may result in a shift-reduce conflict.
- ❑ A state may include two different final items. This might result in reduce-reduce conflict.

11. Explain viable prefixes.

Ans: The stack in a shift-reduce parser for a grammar G can hold the prefixes of right-sentential form. However, not all the prefixes of right sentential forms can appear in the stack, that is, the stack is allowed to hold only some of the prefixes. These prefixes that the stack can hold are known as **viable prefixes**. A viable prefix is so called because it is always possible to add terminal symbols to the end of a viable prefix to obtain a right-sentential form. The fact that the LR(0) automata recognize viable prefixes forms the basis for SLR parsing. For example, we consider the following item:

$$X \rightarrow \beta_1 \beta_2$$

It is valid for a viable prefix $\alpha\beta_1$ if there exists a derivation

$$S' \xRightarrow[m]{*} \alpha X w \xRightarrow[m]{} \alpha\beta_1\beta_2 w$$

We can deduce the information about whether to take shift action or reduce action from the fact that $X \rightarrow \beta_1\beta_2$ is valid for $\alpha\beta_1$ as follows:

- If $\beta_2 \neq \in$, it indicates that we have not yet shifted the handle onto the stack, so need to perform a shift action.
- If $\beta_2 = \in$, it indicates as if $X \rightarrow \beta_1$ is the handle, and we should reduce by this production.

Thus, it is clear that for the same viable prefix, two valid items may indicate to do different things. Such conflicts can be resolved by looking ahead the next input symbol.

Generally, an item can be valid for many viable prefixes. The set of items valid for a viable prefix γ can be computed by determining the set of items that can be reached from the initial state along the path labeled γ in the LR(0) automaton for the grammar.

12. What is the canonical LR parser?

Ans: A canonical LR (CLR) parser is more powerful than LR parser as it makes full use of one or more lookahead symbols. It contains some extra information in the form of a terminal symbol, as a second component in each item of state. Thus, in CLR parser, an item can be described as follows:

$$[A \rightarrow \alpha.\beta, a]$$

where

$A \rightarrow \alpha\beta$ is a production, and
 a is the terminal symbol or right end marker \$.

Such an item is defined as **LR(1) item**, where 1 refers to the length of second component, called the **lookahead** of the item. If $\beta \neq \in$, then the lookahead does not effect the item $[A \rightarrow \alpha.\beta, a]$. However, if the item has the form $[A \rightarrow \alpha., a]$, then it calls for a reduction by $A \rightarrow \alpha$ only if the next input symbol is a . That is, we are compelled to reduce $A \rightarrow \alpha$ only on those input symbols a for which $[A \rightarrow \alpha., a]$ is an LR(1) item in the state on the top of the stack.

13. Write the algorithm for computation of sets of LR(1) items.

Or

Define CLOSURE (I) and GOTO (I, X) functions for LR(1) grammar.

Ans: The algorithm for computing the sets of LR(1) items is basically the same as that of the canonical sets of LR(0) items—only the procedures for computing the CLOSURE and GOTO need to be modified as shown in Figure 5.3.

In Figure 5.3, the function `items()` is the main function that calls the CLOSURE and GOTO functions for constructing the sets of LR(1) items for grammar G' .

```

procedure CLOSURE(I)
  Begin
    Do
      For (each item  $[A \rightarrow \alpha.B\beta, a]$  in I)
        For (each production  $B \rightarrow \gamma$  in  $G'$ )
          For (each terminal  $b$  in FIRST( $\beta a$ ))

```

```

        add [B → . γ, b] to I
    While there are some items to be added to set I
    return I
End

procedure GOTO(I, X)
Begin
    Initialize J to be the empty set
    For (each item [A → α.Xβ, a] in I)
        add item [A → α.Xβ, a] to set J
    return CLOSURE(J)
End

void items(G')
Begin
    C = CLOSURE([S' → .S, $])
    Do
        For (each set of items I in C)
            For (each grammar symbol X)
                If (GOTO(I, X) is not empty and not in C)
                    add GOTO(I, X) to C
        While there are some sets of items to be added to C
    End
End

```

Figure 5.3 Computation of Sets of LR(1) Items for Grammar G'

14. Give the algorithm for the construction of canonical LR parsing table.

Ans: Canonical LR parsing tables are constructed by the LR(1) ACTION and GOTO functions from the set of LR(1) items. The ACTION and GOTO entries are constructed in the parsing table using the following algorithm:

Step 1: For any augmented grammar G' , construct the collection of sets of LR(0) items $C' = \{I_0, I_1, \dots, I_n\}$.

Step 2: For each state in C , construct a row in CLR table and name the rows from 0 to n . Partition the columns into ACTION and GOTO, where ACTION will have all terminal symbols of grammar G along with symbol $\$,$ and GOTO will have all the non-terminals of G .

Step 3: For each I_i construct a state i . The action entries for state i in CLR parsing table are determined using the following rules:

- If $[A \rightarrow \alpha.a\beta, b]$ is in I_i and $\text{GOTO}(I_i, a) = I_j$, where a is a terminal, then $\text{ACTION}[i, a] = \text{"shift } j\text{"}$.
- If $[A \rightarrow \alpha., a]$ is in I_i , and $A \neq S'$ then $\text{ACTION}[i, a] = \text{"reduce } A \rightarrow \alpha.\text{"}$.
- If $[S' \rightarrow S., \$]$ is in I_i , then $\text{ACTION}[i, \$] = \text{"accept"}.$

If any conflicting actions occur from the above rules, we will not consider the grammar as LR(1). In that case, this algorithm will not be able to produce a valid parser.

Step 4: The goto entries for state i in the CLR parsing table can be determined using the following rule:

If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}[i, A] = j$.

Step 5: The undefined entries are made 'error' entries.

Step 6: The state of parser that is constructed from the set of items containing $[S' \rightarrow .S, \$]$ is the initial state of the parser.

The table containing the parsing action and goto functions produced by the canonical LR parsing table algorithm is called the **canonical LR(1) parsing table**, and the LR parser that uses this table is called a **canonical LR(1) parser**. If there are no multiple defined entries in the parsing action function, then the given grammar is called an LR(1) grammar.

15. What is LALR parsing? Give the algorithm for the construction of LALR parsing table.

Ans: LALR parsers or **lookahead LR** parsers are specialized form of LR parsers that work on most of the programming language and can be implemented more efficiently. LALR parsers lie in between SLR parsers and canonical LR parsers in terms of power of parsing grammars. That is, they deal with more grammars than that of SLR parsers but less than that of LR(1) parsers. LALR parsing technique is most frequently used because the tables generated by it are considerably smaller than the canonical LR tables. Some of the most commonly used parser generators programs such as YACC and bison use the LALR parsing technique for constructing the parsing table.

The LALR parsing table can be constructed from the collection of merged set of items. The main principle behind the construction of LALR parsing table is to merge the set of LR(1) items having the same core to form the condensed set of LR(1) items. The sets containing the same first components of the LR(1) items are said to have the same **core**. That is, the components appearing at the left hand side of the dot symbol in a production are same; only the lookahead symbol(s) (the symbol appearing at the right hand side of the dot symbol) varies. Thus, the number of sets of items produced by LALR method is smaller than the number of sets produced by the CLR method. The algorithm for the construction of LALR parsing table is shown below:

Step 1: For any augmented grammar G' , construct the collection of sets of LR(1) items $C = \{I_0, I_1, I_2, \dots, I_n\}$.

Step 2: For each core present among the sets of LR(1) items, determine the sets that have that core and replace them with their union. Suppose the resulting sets of LR(1) items be $C' = \{J_0, J_1, J_2, \dots, J_k\}$, where J_i is a union of one or more sets of LR(1) items having the same core. Mathematically, $J_i = \{I_1 \cup I_2 \cup I_3 \cup \dots \cup I_m\}$.

Step 3: For each J_i construct a state i . The action entries for state i in LALR parsing table are determined in the same way as that of canonical LR parser. If any conflicting actions occur, we will not consider the grammar as LALR(1). In that case, this algorithm will not be able to produce a valid parser.

Step 4: The goto table is constructed by taking the union of all sets of items having common core. Since $I_1, I_2, \dots, I_m$ all have the common core, the cores of $\text{GOTO}(I_1, X), \text{GOTO}(I_2, X), \dots, \text{GOTO}(I_m, X)$ will also be same. If we consider U as the union of all sets of items having the common core as $\text{GOTO}(I_1, X)$, then $\text{GOTO}(J, X) = U$, where $J = I_1 \cup I_2 \cup \dots \cup I_m$.

16. Specify the merits and demerits of LALR parser.

Ans: The merits of LALR parser are as follows:

- The LALR parser tables are smaller than the CLR tables.
- An LALR grammar can easily express the syntactic structure of programming languages.
- The LALR parsing technique can be implemented more efficiently.

- The LALR parser provides a good trade off between power and efficiency.
- Merging of items never introduces shift-reduce conflict unless the conflict is already present in LR(1) configuration sets.

The demerits of LALR parser are as follows:

- The construction of parser table from the collection of LR(1) items requires too much space and time.
- Merging of items may introduce reduce-reduce conflict. In case reduce-reduce conflict arises, the grammar is not considered as LALR(1).

17. Differentiate between SLR and LALR.

Or

Why LALR parser is considered over SLR?

Ans: In LALR parsing, the reduce entries are made using lookahead sets whereas in SLR, reduce entries are made using succeed sets. The **lookahead set** for LR(0) item $\mathcal{I}$ consists of only those symbols that are expected to be appeared after $\mathcal{I}$'s right hand side has been parsed. On the other hand, the **succeed set** consists of all those symbols that are supposed to appear after $\mathcal{I}$'s left hand side non-terminal. The lookahead set is more specific to parsing context and provides a finer distinction than the succeed set. In SLR parsing, shift-reduce conflict arises whereas merging of items does not introduce any shift-reduce conflict in LALR parsing. Reduce-reduce conflict may occur in LALR parsing.

18. Discuss how YACC can be used to generate a parser?

Ans: YACC stands for **yet another compiler-compiler**. It is an LALR parser generator which is basically available as a command on UNIX system. The first version of YACC was created by S.C. Johnson in early 1970s. It is a tool that compiles a source program and translates it into a C program that implements the parser.

For example, consider a file `translate.y`. The YACC compiler converts this file into a C program `y.tab.c` using the LALR algorithm. The program `y.tab.c` is basically a representation of LALR parser written in C language. This program is then compiled along with the `ly` library to generate the object code `a.out`, which then performs the translation specified by the original YACC program. An input/output translator constructed using YACC is given in Figure 5.4.

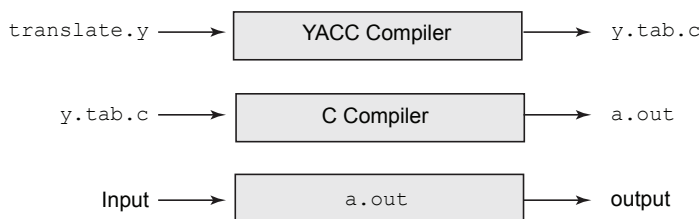


Figure 5.4 Input/Output Translator with YACC

A YACC source program consists of the following three parts:

```

declaration
%%
transition rules
%%
supporting C routines
  
```

- **Declaration section:** This section consists of two optional sections. The first section contains ordinary C declarations delimited by `%{` and `%}`. For example, it may contain `#include` pre-processors as given below:

```
%{
#include<conio.h>
%}
```

The second section contains the declarations of grammar tokens. For example, the following statement declares `DIGIT` as a token:

```
%token DIGIT
```

- **Translation rules section:** This section includes the grammar productions along with their semantic actions. For example, the productions of the following form:

$$\text{head} \rightarrow \text{alternative}_1 \mid \text{alternative}_2 \mid \dots \mid \text{alternative}_n$$

can be written in YACC as follows:

```
head :    alternative1 {semantic action 1}
      |    alternative2 {semantic action 2}
      . . .
      |    alternativen {semantic action n}
      ;
```

The semantic action includes the values associated with the non-terminals of the head. The symbol `$$` is used to refer these values. For example, consider the productions,

$$E \rightarrow E * F \mid F$$

The YACC specification for the above productions can be written as follows:

```
expr : expr '*' factor  {$$ = $4 * $5;}
      | factor
      ;
```

- **Supporting C-routines section:** This section includes the lexical analyzer `yylex()` that produces pairs consisting of token name and their associated values. The attribute values are communicated to the parser through the variable `yylval` already defined in YACC.

19. Explain ambiguity in LR parsers and the ways to eliminate it.

Ans: LR parser involves two conflicts, namely, *shift-reduce conflict* and *reduce-reduce conflict*.

- **Shift-reduce conflict:** This conflict occurs when it is difficult to decide whether to shift a token or to reduce a handle. For example, consider the dangling-else grammar:

```
statement → if condition then statement |
           if condition then statement else statement |
           other statement
```

Suppose the status of the stack and next input symbol at some point is as follows:

Stack	Input
... if condition then statement	else ...\$

Depending on what follows the **else** on the input, the parser may choose to *reduce if* condition **then** statement to statement, or it may choose to *shift else* and then look for another statement to complete the alternative **if** condition **then** statement **else** statement. This gives rise to shift-reduce conflict since the parser cannot decide whether to shift **else** onto the stack or to reduce **if** condition **then** statement.

This ambiguity can be resolved by matching each **else** with its just preceding unmatched **then**. Thus, in our case the next action would be shift **else** onto the stack because it is associated with the previous **then**.

- **Reduce-reduce conflict:** This conflict occurs when we know we have a handle but the next input symbol and the stack's contents are not enough to determine which production is to be used in a reduction. For example, consider a language in which a procedure can be invoked by giving the procedure name along with the parameters surrounded by parentheses, and array references are also made using the same syntax. Some of the productions for the grammar of our language would be as follows:

statement $\rightarrow$ id (parameter_list)	(1)
statement $\rightarrow$ expression: = expression	(2)
parameter_list $\rightarrow$ parameter_list, parameter	(3)
parameter_list $\rightarrow$ paramater	(4)
parameter $\rightarrow$ id	(5)
expression $\rightarrow$ id (expression_list)	(6)
expression_list $\rightarrow$ expression_list, expression	(7)
expression_list $\rightarrow$ expression	(8)
expression $\rightarrow$ id	(9)

Let us consider an input string A(X, Y). The token stream for the given input string for the parser is **id**(**id**, **id**). Now, the configuration of the parser after shifting the initial three tokens onto the stack is as follows:

Stack	Input
... id (id	, id)...\$

It is clear that we need to reduce the **id** that is on top of the stack, but it is not clear that which production needs to be used for reduction. If A is a procedure name then production (5) needs to be used, and if A is an array then production (9) needs to be followed. Thus, reduce-reduce conflict occurs.

This conflict can be resolved by changing the token **id** in production (1) by **procid**, and by using a more sophisticated lexical analyzer that returns token **procid** for an identifier which is a procedure name, and **id** for an array name. Before returning a token, the lexical analyzer needs to consult the symbol table. Now, if A is a procedure then after this modification of token stream the configuration of the parser would be:

Stack	Input
... procid (id	, id)...\$

20. Explain error recovery in LR parsing.

Ans: An LR parser detects an error when it finds undefined entries in the LR parsing table or when an empty entry for an input combination is made in the table. Checking of `goto` tables never results in errors. The requirement of error recovery is to enable the parser to detect syntax errors and reporting them to the user for correction. There are two error recovery strategies in LR parsing, namely, *panic mode* and *phrase level error recovery*.

- ❑ **Panic mode error recovery:** The panic mode error recovery can be implemented in LR parsing in the following manner:
 - The stack is scanned down until a state s having a `goto` on a particular non-terminal X is found. The non-terminal would be an expression or statement.
 - Then we discard zero or more input symbols until we found a symbol x that follows X . If X is a statement, then x would be a semicolon or end.
 - Parser then pushes the state $GOTO[s, X]$ onto the stack and resumes normal parsing.
- ❑ **Phrase level error recovery:** The phrase level error recovery can be implemented in LR parsing in the following manner:
 - Each error entry in the LR parsing table is examined and on the basis of language usage, the most likely programmer error that would result in that error is determined.
 - An appropriate error recovery procedure is then constructed. deemed
 - For each error entry, the top of the stack or the first input symbols, or both are then modified. For example, a comma is replaced by a semicolon or a missing semicolon is inserted or an extraneous semicolon is deleted.

21. Construct an LR(0) parsing table for the following grammar G:

$P \rightarrow Q)$
 $P \rightarrow Q, R \mid (R, R$
 $R \rightarrow \{num, num\}$

Ans: The augmented grammar G' for the above grammar G is:

$P' \rightarrow P$ (rule 0)
 $P \rightarrow Q)$ (rule 1)
 $P \rightarrow Q, R$ (rule 2)
 $P \rightarrow (R, R$ (rule 3)
 $R \rightarrow \{num, num\}$ (rule 4)

Item set number 0, I_0

$P' \rightarrow .P$
 $+P \rightarrow .Q)$
 $+P \rightarrow .Q, R$
 $+P \rightarrow .(R, R$

where '+' denotes the closure of the item $P' \rightarrow .P$ and it is not any terminal. In I_0 symbols just after the dot are $P, Q,$ and $($.

Item set number 1, I_1 (for the symbol P of I_0)

$P' \rightarrow P.$

Item set number 2, I_2 (for the symbol Q of I_0)

$$P \rightarrow Q \ . \)$$

$$P \rightarrow Q \ . , \ R$$
Item set number 3, I_3 (for the symbol $($ of I_0)

$$P \rightarrow (\ . \ R, \ R$$

$$+R \rightarrow . \ \{num, \ num\}$$

In I_1 , no symbol is left after the dot.

In I_2 , symbols just after the dot are $)$ and $, \ .$

Item set number 4, I_4 (for the symbol $)$ of I_2)

$$P \rightarrow Q \) \ .$$
Item set number 5, I_5 (for the symbol $,$ of I_2)

$$P \rightarrow Q, \ . \ R$$

$$+R \rightarrow . \ \{num, \ num\}$$

In I_3 , symbols just after the dot are R and $\{.$

Item set number 6, I_6 (for the symbol R of I_3)

$$P \rightarrow (\ R \ . \ , \ R$$
Item set number 7, I_7 (for the symbol $\{$ of I_3)

$$R \rightarrow \{ . \ num, \ num\}$$

In I_4 , no symbol is left after the dot.

In I_5 , symbols just after the dot are R and $\{.$

Item set number 8, I_8 (for the symbol R of I_5)

$$P \rightarrow Q, \ R \ .$$

Whereas, symbol $\{$ of I_5 is already processed in I_7 .

In I_6 , symbol just after the dot is $, \ .$

Item set number 9, I_9 (for the symbol $,$ of I_6)

$$P \rightarrow (\ R, \ . \ R$$

$$+R \rightarrow . \ \{num, \ num\}$$

In I_7 , symbol just after the dot is num .

Item set number 10, I_{10} (for the symbol num in I_7)

$$R \rightarrow \{num \ . \ , \ num\}$$

In I_8 , no symbol is left after the dot.

In I_9 , symbols just after the dot are R and $\{.$

Item set number 11, I_{11} (for the symbol R of I_9)

$$P \rightarrow (\ R, \ R \ .$$

Also symbol { of I_9 is already processed in I_7 .

In I_{10} , symbol after the dot is , .

Item set number 12, I_{12} (for the symbol , of I_{10})

$$R \rightarrow \{ \text{num}, \text{ . num} \}$$

In I_{11} , no symbol is left after the dot.

In I_{12} , symbol after the dot is num.

Item set number 13, I_{13} (for the symbol N of I_{12})

$$R \rightarrow \{ \text{num}, \text{ num . } \}$$

In I_{13} , symbol after the dot is }.

Item set number 14, I_{14} (for the symbol } of I_{13})

$$R \rightarrow \{ \text{num}, \text{ num} \}.$$

Now, in I_{14} , no symbol is left after the dot.

Thus, the transition table becomes:

Item Set	)	'	(	{	Num	}	P	Q	R
0			3				1	2	
1									
2	4	5							
3				7					6
4									
5				7					8
6		9							
7					10				
8									
9				7					11
10		12							
11									
12					13				
13						14			
14									

Thus, the action/goto table becomes:

Item Set	Action							Goto		
	)	'	(	{	Num	}	\$	P	Q	R
0			S_3					1	2	
1							acc			
2	S_4	S_5								
3				S_7						6

(Continued)

(Continued)

Item Set	Action							Goto		
	)	'	(	{	Num	}	\$	P	Q	R
4	r_1	r_1	r_1	r_1	r_1	r_1	r_1			
5				S_7						8
6		S_9								
7					S_{10}					
8	r_2	r_2	r_2	r_2	r_2	r_2	r_2			
9				S_7						11
10		S_{12}								
11	r_3	r_3	r_3	r_3	r_3	r_3	r_3			
12					S_{13}					
13						S_{14}				
14	r_4	r_4	r_4	r_4	r_4	r_4	r_4			

This is the resultant LR(0) parsing table.

22. Construct the sets of LR(0) items for the following grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

And also construct the parsing table.

Ans: The given grammar is ambiguous as the precedence or associativity of the operator $*$ and $+$ is not specified. For the given grammar, the augmented grammar is as follows:

$E' \rightarrow E$
 $E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow (E)$
 $E \rightarrow id$

The set of LR(0) items is as follows:

$I_0: E' \rightarrow .E$
 $E \rightarrow .E + E$
 $E \rightarrow .E * E$
 $E \rightarrow .(E)$
 $E \rightarrow .id$
 $I_1: E' \rightarrow E$
 $E \rightarrow E. + E$
 $E \rightarrow E. * E$
 $I_2: E \rightarrow (.E)$
 $E \rightarrow .E + E$
 $E \rightarrow .E * E$
 $E \rightarrow (E)$
 $E \rightarrow .id$
 $I_3: E \rightarrow id.$

$I_4:$ $E \rightarrow E + .E$
 $E \rightarrow .E + E$
 $E \rightarrow .E * E$
 $E \rightarrow .(E)$
 $E \rightarrow .id$

$I_5:$ $E \rightarrow E * .E$
 $E \rightarrow .E + E$
 $E \rightarrow .E * E$
 $E \rightarrow .(E)$
 $E \rightarrow .id$

$I_6:$ $E \rightarrow (E.)$
 $E \rightarrow E. + E$
 $E \rightarrow E. * E$

$I_7:$ $E \rightarrow E + E.$
 $E \rightarrow E. + E$
 $E \rightarrow E. * E$

$I_8:$ $E \rightarrow E * E.$
 $E \rightarrow E. + E$
 $E \rightarrow E. * E$

$I_9:$ $E \rightarrow (E).$

Now, the parsing table for the above set of LR(0) items will be:

State	Action						Goto
	id	+	*	(	)	\$	E
0	S_3			S_2			1
1		S_4	S_5			acc	
2	S_3			S_2			6
3		r_4	r_4		r_4	r_4	
4	S_3			S_2			8
5	S_3			S_2			8
6		S_4	S_5		S_3		
7		r_1	S_5		r_1	r_1	
8		r_2	r_2		r_2	r_2	
9		r_3	r_3		r_3	r_3	

23. Every SLR(1) is unambiguous but there are few unambiguous grammars that are not SLR(1). Verify this for the following productions.

$S \rightarrow L = R$

$S \rightarrow R$

$L \rightarrow * R$

$L \rightarrow id$

$R \rightarrow L$

Ans: The augmented grammar G' for the above productions is as follows:

$$\begin{aligned}
S' &\rightarrow S \\
S &\rightarrow L = R \\
S &\rightarrow R \\
L &\rightarrow *R \\
L &\rightarrow \text{id} \\
R &\rightarrow L
\end{aligned}$$

The canonical collection of LR(0) items for G are as follows:

Starting with closure ($S' \rightarrow .S$), we get I_0

$$\begin{aligned}
S' &\rightarrow .S \\
S &\rightarrow .L = R && \text{by Rule 1 of closure, } A \rightarrow \alpha . B\beta \\
S &\rightarrow .R && B \rightarrow .\gamma \\
L &\rightarrow .*R && \text{closure(1), by Rule 3, } B \rightarrow .\gamma \\
L &\rightarrow .\text{id} && B \rightarrow .\gamma \text{ in closure(2)} \\
R &\rightarrow .L && B \rightarrow .\gamma \text{ closure(2)}
\end{aligned}$$

$$\begin{aligned}
I_1 &= \text{GOTO}(I_0, S) \\
&= \text{Closure}(S' \rightarrow S.), \text{ we obtain} \\
&S' \rightarrow S.
\end{aligned}$$

$$\begin{aligned}
I_2 &= \text{GOTO}(I_0, L) \\
&= \text{Closure}(S \rightarrow L . = R) \cup \text{Closure}(R \rightarrow L.), \text{ we obtain} \\
S &\rightarrow && L. = R \\
R &\rightarrow && L.
\end{aligned}$$

$$\begin{aligned}
I_3 &= \text{GOTO}(I_0, R) \\
&= \text{Closure}(S \rightarrow R.), \text{ we obtain} \\
S &\rightarrow R.
\end{aligned}$$

$$\begin{aligned}
I_4 &= \text{GOTO}(I_0, *) \\
&= \text{Closure}(L \rightarrow *.R), \text{ we obtain} \\
L &\rightarrow *.R \\
R &\rightarrow .L && B \rightarrow .\gamma \text{ in closure } (L \rightarrow *.R) \\
L &\rightarrow .*R && B \rightarrow .\gamma \text{ in closure } (R \rightarrow .L) \\
L &\rightarrow .\text{id} && B \rightarrow .\gamma \text{ in closure } (L \rightarrow .\text{id})
\end{aligned}$$

$$\begin{aligned}
I_5 &= \text{GOTO}(I_0, \text{id}) \\
&= \text{Closure}(B \rightarrow \text{id}.), \text{ we obtain} \\
L &\rightarrow \text{id}.
\end{aligned}$$

$$\begin{aligned}
I_6 &= \text{GOTO}(I_2, =) \\
&= \text{Closure}(S \rightarrow L = .R), \text{ we obtain} \\
S &\rightarrow L = .R \\
R &\rightarrow .L && B \rightarrow .\gamma \text{ in closure } (S \rightarrow L = .R) \\
L &\rightarrow .*R && B \rightarrow .\gamma \text{ in closure } (R \rightarrow .L) \\
L &\rightarrow .\text{id} && B \rightarrow .\gamma \text{ in closure } (R \rightarrow .L)
\end{aligned}$$

$$\begin{aligned}
I_7 &= \text{GOTO}(I_4, R) \\
&= \text{Closure}(L \rightarrow *.R.), \text{ we obtain} \\
L &\rightarrow *.R.
\end{aligned}$$

$I_8 = \text{GOTO}(I_4, L)$
 $= \text{Closure}(R \rightarrow L.),$ we obtain
 $R \rightarrow L.$
 $I_9 = \text{GOTO}(I_6, R)$
 $= \text{Closure}(S \rightarrow L = R.),$ we obtain
 $S \rightarrow L = R.$

Thus, we get the canonical LR(0) items, now to verify whether this grammar is SLR(1) or not by applying rule (3) of SLR parsing table algorithm.

- ❑ Consider the production, $S \rightarrow L. = C$ in I_2 .
- ❑ Compare it with $A \rightarrow \alpha. a\beta$, we obtain that a is $=$.
- ❑ We know that $\text{GOTO}(I_2, =) = I_6$; therefore, by applying rule 3(a) of SLR parser table algorithm, we obtain $\text{ACTION}(2, =) = S_6$.
- ❑ But one more production exists in I_2 , that is, $R \rightarrow L.$ By applying rule 3(b) of SLR table algorithm, and comparing $R \rightarrow L.$ with $A \rightarrow \alpha.$ and $\text{FOLLOW}(R)$ contains $'='$, therefore, $\text{ACTION}(2, =) = r_5$, that is, reduce by $R \rightarrow L.$
- ❑ Thus, we have two actions shift and reduce for $(2, =)$ in SLR table, which means shift-reduce conflict occurs. Therefore, the grammar is not SLR(1), even if the grammar is unambiguous.

The parsing table for the above grammar is designed below:

State	Action				Goto		
	id	*	=	\$	S	L	R
0	S_5	S_4			1	2	3
1				acc			
2			$S_6 r_5$	r_5			
3				r_2			
4	S_5	S_4				8	7
5			r_4	r_4		8	9
6	S_5	S_4					
7			r_3	r_3			
8			r_5	r_5			
9				r_1			

24. Consider the following grammar G:

$E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow id$

- (i) List the canonical collection of sets of LR(0) items for the given grammar.
- (ii) Construct SLR parsing table for the grammar.
- (iii) Show the moves of the parser for the input string $id * id + id$.

Ans: The augmented grammar G' for the above grammar G will be

$$\begin{aligned} E' &\rightarrow E \\ E &\rightarrow E + T \\ E &\rightarrow T \\ T &\rightarrow T * F \\ T &\rightarrow F \\ F &\rightarrow (E) \\ F &\rightarrow id \end{aligned}$$

(i) The item sets for the new grammar G' will be determined as follows:

Item set number 0:

$$\begin{aligned} E' &\rightarrow \cdot E \\ +E &\rightarrow \cdot E + T \\ +E &\rightarrow \cdot T \\ +T &\rightarrow \cdot T * F \\ +T &\rightarrow \cdot F \\ +F &\rightarrow \cdot (E) \\ +F &\rightarrow \cdot id \end{aligned}$$

In I_0 , symbols just after the dot are $E, T, F, (, id$.

Item set number 1, I_1 (for the symbol E of I_0), we have:

$$\begin{aligned} E' &\rightarrow E \cdot \\ E &\rightarrow E \cdot + T \end{aligned}$$

Item set number 2, I_2 (for the symbol T of I_0), we have:

$$\begin{aligned} E &\rightarrow T \cdot \\ T &\rightarrow T \cdot * F \end{aligned}$$

Item set number 3, I_3 (for the symbol F of I_0), we have:

$$T \rightarrow F \cdot$$

Item set number 4, I_4 (for the symbol $($ of I_0), we have:

$$\begin{aligned} F &\rightarrow (\cdot E) \\ +E &\rightarrow \cdot E + T \\ +E &\rightarrow \cdot T \\ +T &\rightarrow \cdot T * F \\ +T &\rightarrow \cdot F \\ +F &\rightarrow \cdot (E) \\ +F &\rightarrow \cdot id \end{aligned}$$

Item set number 5, I_5 (for the symbol id of I_0), we have:

$$F \rightarrow id \cdot$$

In I_1 , symbol just after the dot is $+$.

Thus, **item set number 6, I_6** (for the symbol $+$ of I_1), we have:

$$\begin{aligned}
 E &\rightarrow E + \cdot T \\
 +T &\rightarrow \cdot T * F \\
 +T &\rightarrow \cdot F \\
 +F &\rightarrow \cdot (E) \\
 +F &\rightarrow \cdot id
 \end{aligned}$$

In I_2 , symbol after the dot is $*$

Thus, **item set number 7**, I_7 (for the symbol ' $*$ ' of I_2), we have:

$$\begin{aligned}
 T &\rightarrow T * \cdot F \\
 +F &\rightarrow \cdot (E) \\
 +F &\rightarrow \cdot id
 \end{aligned}$$

In I_3 , there is no symbol after the dot.

In I_4 , symbols after the dot are $E, T, F, (, id$.

Here,

T is already processed in I_2 .

F is already processed in I_3 .

$($ is already processed in I_4

id is already processed in I_5 .

Thus, **item set number 8**, I_8 (for the symbol E of I_4), we have:

$$F \rightarrow (E \cdot)$$

In I_5 , there is no symbol to be processed after the dot.

In I_6 , symbols after the dot are $T, F, (, id$.

Here,

F is already processed in I_3 .

$($ is already processed in I_4

id is already processed in I_5 .

Thus, **item set number 9**, I_9 (for the symbol T of I_6), we have:

$$E \rightarrow E + T \cdot$$

In I_7 , symbols after the dot are $F, (, id$.

Here,

$($ is already processed in I_4

id is already processed in I_5 .

Thus, **item set number 10**, I_{10} (for the symbol F of I_7), we have:

$$T \rightarrow T * F \cdot$$

In I_8 , symbol after the dot is $)$.

Thus, **item set number 11**, I_{11} (for the symbol $)$ of I_8), we have:

$$F \rightarrow (E) \cdot$$

Now, in I_9, I_{10}, I_{11} there is no symbol left after the dot.

Therefore, the transition table will be:

Item Set	id	+	*	(	)	E	T	F
0	5			4		1	2	3
1		6						
2			7					
3								
4	5			4		8	2	3
5								
6	5			4			9	3
7	5			4				10
8					11			
9								
10								
11								

Now, the action/goto table will be designed as given below:

State	Action						Goto		
	id	+	*	(	)	\$	E	T	F
0	S_5			S_4			1	2	3
1		S_6				acc			
2	r_2	r_2	S_7, r_2	r_2	r_2	r_2			
3	r_4	r_4	r_4	r_4	r_4	r_4			
4	S_5			S_4			8	2	3
5	r_6	r_6	r_6	r_6	r_6	r_6			
6	S_5			S_4				9	3
7	S_5			S_4					10
8					S_{11}				
9	r_1	r_1	r_1	r_1	r_1	r_1			
10	r_3	r_3	r_3	r_3	r_3	r_3			
11	r_5	r_5	r_5	r_5	r_5	r_5			

It is clear from the table that action entry for state 2 contains shift-reduce conflict; thus, it is not LR(0).

(ii) **For SLR parsing table:**

$\text{FOLLOW}(E) = \{\$ \}$ as it is the start symbol

$\text{Follow}(E) = \text{FIRST}(+T) = \{+\}$

$\text{Follow}(E) = \text{FIRST}()) = \{) \}$

Therefore, $\text{FOLLOW}(E) = \{+,), \$ \}$

And in state 2, reduction r_2 is valid only in the columns $\{+,), \$ \}$

Now, $\text{FOLLOW}(T) = \text{FOLLOW}(E) = \{+,), \$ \}$

$\text{FOLLOW}(T) = \text{FIRST}(*F) = \{*\}$

Therefore, $\text{FOLLOW}(T) = \{+, *,), \$ \}$

And in state 3, reduction r_4 is valid only in the columns $\{+, *,), \$ \}$

Now, $\text{FOLLOW}(F) = \text{FOLLOW}(T) = \{+, *,), \$ \}$

Therefore, in state 5 reduction r_6 is valid only in the columns $\{+, *,), \$\}$, in state 9 r_1 is valid only in the columns $\{+,), \$\}$, in state 10 r_3 is valid only in the columns $\{+, *,), \$\}$ and in state 11 r_5 is valid only in the columns $\{+, *,), \$\}$.

Now, after solving shift-reduce conflict, the SLR parsing table will be:

State	Action						Goto		
	id	+	*	(	)	\$	E	T	F
0	S_5			S_4			1	2	3
1		S_6				Acc			
2		r_2	S_7		r_2	r_2			
3		r_4	r_4		r_4	r_4			
4	S_5			S_4			8	2	3
5		r_6	r_6		r_6	r_6			
6	S_5			S_4				9	3
7	S_5			S_4					10
8		S_6				S_{11}			
9		r_1	S_7		r_1	r_1			
10		r_3	r_3		r_3	r_3			
11		r_5	r_5		r_5	r_5			

(iii) Moves of parser to accept the input string $id * id + id$ are shown below:

Stack	Input	Action of Parser
[0]	$id * id + id \$$	Shift '0' initially
[0 id 5]	$* id + id \$$	Action (0, id) = S_5
[0 F 3]		Action (5, *) = r_6 Goto (0, F) = 3
[0 T 2]	$(id + id \$$	Action (3, *) = r_4 Goto (0, T) = 2
[0 + 2 * 7]	$id + id \$$	Action (2, *) = S_5
[0 T 2 * 7 id 5]	$+ id \#$	Action (7, id) = S_5
[0 T 2 * 7 F 10]	$+ id \$$	Action (5, +) = r_6 Goto (7, F) = 10
[0 T 2]	$+ id \$$	Action (10, +) = r_3 Goto (0, T) = 2
[0 E 1]	$+ id \$$	Action (2, +) = r_2 Goto (0, E) = 1
[0 E 1 + 6]	$id \$$	Action (1, +) = S_6
[0 E 1 + 6 F 3]	$\$$	Action (5, \\$) = r_6 Goto (6, F) = 3
[0 E 1 + 6 T 9]	$\$$	Action (3, \\$) = r_4 Goto (6, T) = 9
[0 E 1]	$\$$	Action (9, \\$) = r_1 Goto (0, E) = 1
		"Acc" because Action (1, \\$) = "acc"

25. Construct the LR(1) items and the CLR parsing table for the following grammar:

$S \rightarrow CC$
 $C \rightarrow cC$
 $C \rightarrow d$

Ans: The augmented grammar G' for the above grammar will be:

$S' \rightarrow S$
 $S \rightarrow CC$
 $C \rightarrow cC$
 $C \rightarrow d$

Apply ITEMS (G') procedure to construct LR(1) items.

For I_0 , Closure ($S' \rightarrow .S, \$$)

S'	$\rightarrow$	ϵ	$\cdot$	S	ϵ	$,$	$\$$
A	$\rightarrow$	α	$\cdot$	B	β	$,$	a

Here $B = S, \beta = \epsilon$

$FIRST(\$) = \{\$ \}$ and we have the production $S \rightarrow CC$ which is of the form $B \rightarrow \gamma$ so add $[B \rightarrow .\gamma, b]$ to I , for each b in $FIRST(\beta\gamma)$, that is, $S \rightarrow .CC, \$$ and then again computing the closure.

Closure [$S \rightarrow .CC, \$$]

S	$\rightarrow$	ϵ	$\cdot$	C	C	$,$	$\$$
A	$\rightarrow$	α	$\cdot$	B	β	$,$	a

Here $B = C$ and $\beta = C, a = \$$

$FIRST(\beta a) = FIRST(C\$) = \{c, d\}$ and we have $B \rightarrow \gamma$, that is, $C \rightarrow cC$ and $C \rightarrow d$ so, we add the following productions for each $FIRST(\beta a)$:

$C \rightarrow .cC, c$
 $C \rightarrow .cC, d$
 $C \rightarrow .d, c$
 $C \rightarrow .d, d$

Now, we can write LR(1) items for I_0 as:

$S' \rightarrow .S, \$$
 $S \rightarrow .CC, \$$
 $C \rightarrow .cC, c/d$
 $C \rightarrow .d, c/d$

$I_1 = \text{GOTO}(I_0, s)$
 $= \text{Closure}(S' \rightarrow S., \$)$

Thus, I_1 will have: $S' \rightarrow S., \$$

$I_2 = \text{GOTO}(I_0, C)$
 $= \text{Closure}(S \rightarrow C.C, \$)$

Thus, the productions in I_2 will be:

$$\begin{aligned} S &\rightarrow C.C, \$ \\ C &\rightarrow c.C, \$ \quad [B \rightarrow .\gamma, a] \text{ of closure}(1) \\ C &\rightarrow .d, \$ \quad [B \rightarrow .\gamma, a] \text{ of closure}(1) \end{aligned}$$

$$\begin{aligned} I_3 &= \text{GOTO}(I_0, c) \\ &= \text{Closure}(C \rightarrow c.C, c/d) \end{aligned}$$

Thus, the productions in I_3 will be:

$$\begin{aligned} C &\rightarrow c.C, c/d \\ C &\rightarrow .c.C, c/d \quad [B \rightarrow .\gamma, a] \text{ of closure}(1) \\ C &\rightarrow .d, c/d \quad [B \rightarrow .\gamma, a] \text{ of closure}(1) \end{aligned}$$

$$\begin{aligned} I_4 &= \text{GOTO}(I_0, d) \\ &= \text{Closure}(C \rightarrow d., c/d) \end{aligned}$$

Thus I_4 will have: $C \rightarrow d., c/d$

There is no transition from I_1 .

$$\begin{aligned} I_5 &= \text{GOTO}(I_2, C) \\ &= \text{Closure}(S \rightarrow CC., \$) \end{aligned}$$

Thus, I_5 will have:

$$S \rightarrow CC., \$$$

$$\begin{aligned} I_6 &= \text{GOTO}(I_2, c) \\ &= \text{Closure}(C \rightarrow c.C, \$) \end{aligned}$$

Thus, the productions in I_6 will be:

$$\begin{aligned} C &\rightarrow c.C, \$ \\ C &\rightarrow .c.C, \$ \\ C &\rightarrow .d, \$ \end{aligned}$$

$$\begin{aligned} I_7 &= \text{GOTO}(I_2, d) \\ &= \text{Closure}(C \rightarrow d., \$) \end{aligned}$$

Thus, I_7 will have: $C \rightarrow d., \$$

$$\begin{aligned} I_8 &= \text{GOTO}(I_3, C) \\ &= \text{Closure}(C \rightarrow cC., c/d) \end{aligned}$$

Thus, I_8 will have:

$$C \rightarrow cC., c/d$$

$$\begin{aligned} \text{And } \text{GOTO}(I_3, C) &= \text{Closure}(C \rightarrow c.C, c/d) \\ &= I_3 \end{aligned}$$

$$\begin{aligned} \text{And } \text{GOTO}(I_3, d) &= \text{Closure}(C \rightarrow d., c/d) \\ &= I_4 \end{aligned}$$

No transition on I_4 and I_5 .

$$\begin{aligned} \text{So, } I_9 &= \text{GOTO}(I_6, C) \\ &= \text{Closure}(C \rightarrow cC., \$) \end{aligned}$$

Thus, I_9 will have:

$$C \rightarrow cC., \$$$

And, $GOTO(I_6, c) = \text{Closure}(C \rightarrow c.C, \$)$
 $= I_6$

And, $GOTO(I_6, d) = \text{Closure}(C \rightarrow d., \$)$
 $= I_7$

And finally, there are no transitions from states I_7, I_8, I_9 .

Now, the canonical LR parser table for the above LR(1) items will be designed as follows:

State	Action			GOTO	
	c	d	\$	S	C
0	S_3	S_4		1	2
1			acc		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

26. Discuss algorithms for computation of sets of LR(1) item. Also, show that the following grammar is LR(1) but not LALR(1).

G: $S \rightarrow Aa / bAc / BC / bBa$

A $\rightarrow d$

B $\rightarrow d$

Ans: The augmented grammar G' for the above grammar will be:

$S' \rightarrow S$

$S \rightarrow Aa$

$S \rightarrow bAc$

$S \rightarrow Bc$

$S \rightarrow bBa$

$A \rightarrow d$

$B \rightarrow d$

Item set number 0, I_0 :

$S' \rightarrow .S, \$$

$+S \rightarrow .Aa, \$$

$+S \rightarrow .bAc, \$$

$+S \rightarrow .Bc, \$$

$+S \rightarrow .bBa, \$$

$+A \rightarrow .d, a$

$+N \rightarrow .d, c$

Symbols found in I_0 are S, A, b, B, d

Item set number 1, I_1 (for the symbol S of I_0)

$S' \rightarrow S., \$$

Item set number 2, I_2 (for the symbol A of I_0)

$S \rightarrow A. a, \$$

Item set number 3, I_3 (for the symbol b of I_0)

$S \rightarrow b.AC, \$$

$S \rightarrow b.Ba, \$$

$+A \rightarrow .d, c$

$+B \rightarrow .d, a$

Item set number 4, I_4 (for the symbol B of I_0)

$S \rightarrow B.c, \$$

Item set number 5, I_5 (for the symbol d of I_0)

$A \rightarrow d., a$

$N \rightarrow d., c$

Symbol found in I_2 is a

Item set number 6, I_6 (for the symbol a of I_2)

$S \rightarrow Aa., \$$

Symbols found in I_3 are A, B, d

Item set number 7, I_7 (for the symbol A of I_3)

$S \rightarrow bA.c, \$$

Item set number 8, I_8 (for the symbol B of I_3)

$S \rightarrow bB.a, \$$

Item set number 9, I_9 (for the symbol d of I_3)

$A \rightarrow d., c$

$B \rightarrow d., a$

Symbol found in I_4 is c

Item set number 10, I_{10} (for the symbol c of I_4)

$S \rightarrow Bc., \$$

Symbol found in I_7 is c

Item set number 11, I_{11} (for the symbol c of I_7)

$S \rightarrow bAc., \$$

Symbol found in I_8 is a

Item set number 12, I_{12} (for the symbol a of I_8)

$S \rightarrow bBa., \$$

The action/goto table will be designed as follows:

State	Action					Goto		
	A	b	bc	d	\$	S	A	B
0		S_3		S_5		1	2	4
1					acc			
2	S_6							
3				S_9			7	8
4			S_{10}					
5	r_5		r_6					
6					r_1			
7			S_{11}					
8	S_{12}							
9	r_6		r_5					
10					r_3			
11					r_2			
12					r_4			

Since the table does not have any conflict, it is LR(1).

For LALR(1) table, item set 5 and item set 9 are same.

Thus, we merge both the item sets $(I_5, I_9) = \text{item set } 59, I_{59}$

Now, the resultant parsing table becomes:

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0		S_3		S_5		1	2	4
1					acc			
2	S_6							
3				S_9			7	8
4			S_{10}					
59	r_5, r_6		r_6, r_5					
6					r_1			
7			S_{11}					
8	S_{12}							
10					r_3			
11					r_2			
12					r_4			

Since the table contains reduce-reduce conflict, it is not LALR(1).

Multiple-Choice Questions

- The most common non-backtracking shift-reduce parsing technique is known as _____.
 - LL parsing
 - LR parsing
 - Top-down parsing
 - Bottom-up parsing

2. The simplest LR parsing technique is _____.
 - (a) CLR parser
 - (b) SLR parser
 - (c) LALR parser
 - (d) LL parser
3. $X \rightarrow A \cdot BC$, the given item indicates that _____.
 - (a) a string derivable from ABC is expected next on the input.
 - (b) a string derivable from BC has already been seen and now a string derivable from A is expected on the input.
 - (c) a string derivable from A has already been seen on the input and now a string derivable from BC is expected.
 - (d) the body of the production has already been seen, and now it is time to reduce it to X.
4. Shift-reduce and reduce-reduce conflicts occur in _____.
 - (a) SLR parser
 - (b) LALR parser
 - (c) CLR parser
 - (d) None of these
5. A parser that accommodates some extra information in the form of a terminal symbol, as a second component is known as _____.
 - (a) SLR parser
 - (b) LALR parser
 - (c) CLR parser
 - (d) LL parser
6. If $A \rightarrow \alpha\beta$ is a production and a is terminal symbol or right end marker $\$,$ then LR(1) items will be defined by the production _____.
 - (a) $[A \rightarrow \alpha \cdot \beta, a]$
 - (b) $[A \rightarrow \cdot \alpha\beta, a]$
 - (c) $[A \rightarrow \alpha \cdot \beta a]$
 - (d) $[A \rightarrow a \cdot \beta, \alpha]$
7. _____ parsers are specialized form of LR parsers that lie in between SLR parsers and canonical LR parsers in terms of power of parsing grammars.
 - (a) LALR parser
 - (b) LR(0) parser
 - (c) CLR(1) parser
 - (d) LR(1) parser
8. The automatic tool/tools to generate an LR parser from a given grammar is/are _____.
 - (a) YACC
 - (b) LEX
 - (c) BISON
 - (d) Both (a) and (b)

Answers

1. (b) 2. (b) 3. (c) 4. (a) 5. (c) 6. (a) 7. (a) 8. (d)